

Imperial College London

IMPERIAL COLLEGE LONDON

MSCAI INDIVIDUAL PROJECT

THESIS

DEPARTMENT OF COMPUTING

Multiscale Discrete Generative Models

Author:
Bobbi Halsell

Supervisor:
Prof. Yingzhen Li

PhD Supervisor:
Jacob Si

Contents

1	Introduction	4
1.1	Generative Image Models for Discrete Data	4
1.2	Thesis Motivation and Objectives	5
2	Background	6
2.1	Methods for Image Generation	6
2.1.1	Autoregressive Models	6
2.1.2	Diffusion Models	7
2.1.3	Masked Image Generation	7
2.1.4	Masked Diffusion	8
2.2	Mathematical Framework of Discrete Diffusion Models	8
2.3	Model Architectures	13
2.4	Conditioning in Models	14
2.5	Conditional Generation	15
2.6	Noise Schedule	16
2.7	Self-Attention Masking Mechanics	18
3	VAR Architecture and Baseline	20
3.1	Tokenisation	21
3.1.1	Multi-Scale Tokenisation	21
3.1.2	Multi-Scale VQ-VAE for VAR Learning	21
3.1.3	Transformer Architecture	22
3.1.4	Input Representation and Processing	22
3.1.5	Attention Masking Mechanisms	23
3.1.6	Token Prediction Objective	23
4	The Model	24
4.1	Model Input	24
4.1.1	Teacher Forcing Training	24
4.1.2	Attention Mask	24
4.1.3	Learnable Mask Embedding	26
4.2	Noise Schedule and Masking Probabilities	26
4.2.1	Noise Schedule	26
4.2.2	Loss Weight Behaviour	26
4.2.3	Masking and Unmasking Probabilities	28
4.3	Model Architecture and Conditioning	28
4.3.1	Class and Time Conditioning	28
4.3.2	Positional Encoding	28
4.3.3	Backbone Architecture.	29
4.3.4	Adaptive Layer Normalisation Implementation	30
4.4	Sampling	31

4.5	Implementation and Experiments.	33
5	Results	35
5.1	Experimentaion Set Up	35
5.2	Evaluation Framework and Metrics	35
5.3	Quantitative Metrics	36
5.3.1	Image Comparison	37
5.3.2	Class Conditional Generation	39
5.4	Qualitative Evaluation	39
6	Complexity Analysis	41
6.1	Autoregressive Image Models	41
6.2	Diffusion Models	43
6.3	Proposed Model	44
7	The Repository	46
7.1	System Architecture and Design	46
7.2	Training and Model Checkpointing	47
7.3	Data Pipeline	48
7.4	Configuration Management	49
7.5	Distributed Training	49
7.6	Experimental Monitoring and Logging	49
8	Conclusion	51
8.1	Conclusion and Reflections	51

Chapter 1

Introduction

Data is often naturally represented as either continuous or discrete, and different modalities align more closely with one or the other. Language and images highlight this contrast. Language is one-dimensional, sequential, and unbounded: tokens are read or generated in order, and sequences can grow arbitrarily long. Images, by contrast, are two-dimensional, bounded within a frame, and perceived more holistically. Tokens in language appear naturally discrete, while images appear naturally continuous. Nevertheless, considerable work explores cross-representations, such as mapping text into continuous spaces [1], or treating images as discrete sequences, either directly through pixels in their 8-bit values [2] or via discretised patch tokens as in VQ-VAEs [3].

In continuous domains such as images, generative models benefit from smooth latent spaces that allow interpolation, refinement, and denoising. Discrete domains lack such continuity: progression between tokens in a finite vocabulary is not naturally smooth, complicating hierarchical or residual generation. Masked diffusion over one-hot representations addresses this by defining refinement steps over discrete variables. By contrast, language models often embed discrete tokens into continuous vector spaces for training and inference, capturing semantic relationships, before mapping outputs back to discrete tokens by sampling or argmax.

In this work, we explore modeling discrete data using a hybrid approach that combines the strengths of autoregressive methods with the discrete analogue of diffusion. Autoregressive models excel at capturing sequential dependencies and global structure, while discrete diffusion provides a framework for iterative refinement and hierarchical generation, allowing the model to progressively correct errors or introduce fine-grained details. By introducing an explicit time dimension into the discrete diffusion process, we aim to provide additional context at each refinement step, effectively encoding a trajectory of intermediate states that guides generation in a coarse-to-fine manner. This setup aims to facilitate smooth transitions between discrete states and balance coherence with local diversity.

1.1 Generative Image Models for Discrete Data

Autoregressive models form the foundation of sequence modelling. Each token is predicted in order, conditioned on all previously generated tokens. This matches the unidirectional nature of text and underpins state-of-the-art models such as GPT-3 [4] and LLaMA [5]. In vision, autoregressive models such as VQ-VAE-2 [6] and VQGAN [7] adopt similar strategies, generating tokens sequentially, often following raster-scan patterns. Despite strong performance, such approaches have limitations. Generation can be slow, unidirectional constraints prevent global context usage, and errors made early in the sequence persist throughout.

Masked modelling offers a complementary strategy by reconstructing missing tokens

from context. In language, BERT [8] demonstrates strong contextual understanding, while in images, methods such as MaskGIT [9] enable parallel prediction with iterative refinement. These approaches capture bidirectional dependencies and allow non-sequential updates. However, they require dedicated decoders for generation and do not scale as naturally as autoregressive methods.

Diffusion-based models represent a third method. In continuous domains, diffusion [10, 11, 12] iteratively denoise noise into data, achieving state-of-the-art results in image synthesis. Discrete diffusion extends this framework to token spaces, typically using masking processes that corrupt and then recover sequences. Such models can generate multiple tokens in parallel and refine predictions bidirectionally.

A key distinction emerges between modalities. Continuous data benefits from smooth latent representations, enabling generative models to interpolate and refine naturally. Discrete data lacks this, and modelling requires architectural adaptations, whether through embedding tokens into continuous spaces or designing hierarchical refinements.

1.2 Thesis Motivation and Objectives

This thesis explores a novel hybrid approach to discrete generative modelling that combines hierarchical autoregressive generation with bidirectional masked diffusion processes. The proposed model, DiffusionVAR, maintains the hierarchical coarse-to-fine progression of VAR[13] while introducing diffusion-based refinement within each resolution scale. This design preserves the causal autoregressive relationship between different resolution scales, enables bidirectional context and iterative refinement within each scale through masked diffusion processes. The hierarchical image generation can be decomposed into two distinct but complementary processes, cross-scale progression and intra-scale refinement. The progression from coarse to fine scales benefits from autoregressive conditioning, as higher-resolution details depend on lower-resolution structure. Within each individual scale, tokens can be generated and refined with bidirectional context and enable computation. Autoregression suits the first process by providing structured conditioning across scales, while diffusion suits the second by enabling parallel updates and comprehensive context within each scale. Our model employs a masked diffusion process within each resolution scale conditioned on all previously generated coarser scales. By merging these two paradigms, DiffusionVAR aims to achieve both the efficiency of parallel generation and the controllability of iterative refinement, while preserving the favourable scaling properties observed in hierarchical autoregressive models. The primary contributions of this work span both theoretical understanding and practical implementation of hybrid autoregressive-diffusion models. Our approach explores various methods to incorporate the who methods while balancing computational efficiency. Additionally, we explore conditioning mechanisms that allow control over the generation process dependent on time or class. The remainder of this thesis provides a comprehensive evaluation of DiffusionVAR across multiple dimensions of generative modelling performance.

Chapter 2

Background

2.1 Methods for Image Generation

2.1.1 Autoregressive Models

Autoregressive (AR) models generate data sequentially, producing each token conditioned on those generated previously, and thereby transform high-dimensional joint distributions into a product of simpler conditional distributions. AR models are based on the factorisation of the joint distribution of a sequence into conditionals using the chain rule. Given a sequence of discrete tokens $x = (x_1, x_2, \dots, x_T)$, the likelihood is written as

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}), \quad (2.1)$$

which encodes the assumption that each token depends only on its prefix. Training is carried out through next-token prediction, and generation proceeds sequentially. Applying this principle to images requires imposing a sequential structure on a two-dimensional grid. Early work such as PixelCNN [2] directly factorised over pixels in raster-scan order, enforcing causality through masked convolutions. While effective at modelling local dependencies, this approach was slow to sample and struggled with global consistency due to restricted receptive fields. Later models such as the Image Transformer [14] mitigated some of these issues by treating images as sequences of non-overlapping patches, reducing sequence length and enabling long-range attention. However, the reliance on raster ordering remained, and generation still scaled quadratically with sequence length. They consider a categorical distribution, which treats each RGB channel independently, with 256 possible values per channel. For a 32×32 image, this results in $256 \cdot 3 = 768$ parameters per pixel, corresponding to over 780k output dimensions in total. Alternatively they implement the discretised mixture of logistics capturing both the ordinal structure of pixel intensities and cross-channel dependencies, requiring only 10 parameters per mixture component per pixel. With 10 mixture components, the total output dimension is reducing the total output dimension to approximately 102k for the same image. Quantised autoencoders, such as VQ-VAE [3], were introduced to further reduce this complexity by compressing images into discrete tokens from a learned codebook, reducing the sequence length and output dimensionality while retaining most of the visual information. For example, VQ-VAE-2 [6] represents images across multiple resolutions using a codebook of 512 entries per scale, each with a dimension of 64, for datasets like ImageNet-256 and FFHQ-1024. Autoregressive models operating in this latent space achieved higher resolution synthesis with shorter sequences. However, the autoregressive formulation introduces a mathematical premise violation: the encoder produces feature maps with interdependent vectors $f_{i,j}$, so after quantisation and flattening, the resulting token sequence $(x_1, \dots, x_{hw})$

retains bidirectional correlations. This contradicts the unidirectional dependency structure assumed in autoregression, where x_t should only depend on $(x_1, \dots, x_{t-1})$.

More recently, Visual Autoregressive Modelling (VAR) [13] has proposed a reformulation that avoids flattening altogether. Instead of predicting tokens in raster order, VAR generates a hierarchy of discrete maps across multiple resolutions. Each map r_k is conditioned on all previously generated coarser scales $r_{<k}$, yielding a coarse-to-fine generation process that better respects the spatial structure of images. This multi-scale formulation addresses the premise violation inherent to quantised autoencoders, improves coherence at both local and global levels. It results a lower FID and faster sampling compared to other well-established diffusion or autoregressive alternatives [15, 11, 16, 9, 17]. In this project, I aim to build upon the VAR framework by integrating diffusion training and sampling strategies. By combining the hierarchical, coarse-to-fine structure of VAR with the stochastic denoising dynamics of diffusion models, we hope to further enhance generation quality while maintaining efficiency, and explore the trade-offs.

2.1.2 Diffusion Models

Diffusion models have emerged as a powerful class of generative methods for image synthesis. Denoising Diffusion Probabilistic Models (DDPMs) [10] were the first to introduce this approach, progressively corrupting images with Gaussian noise and learning to reverse the process, producing high-quality, diverse samples at the cost of slow iterative sampling. Later, Denoising Diffusion Implicit Models (DDIMs) [18] and score-based generative models [19, 20] generalised this framework: DDIMs employ deterministic, non-Markovian sampling for faster inference and controllable latent-space manipulations, while score-based models estimate the gradient of the data log-probability, allowing efficient sampling and extensions to discrete domains[21]. Despite their flexibility, these later approaches are less standard in practice due to additional complexity or reduced diversity in generated samples.

Extensions of diffusion models to discrete domains have enabled generative modelling over categorical or quantised data, such as pixel intensities, language and latent tokens.

Diffusion models have been extended to discrete domains, allowing generative modelling over categorical or quantised data, including discretised pixel values, text, and latent tokens. Masked diffusion has similarly been applied to language, formalising the process within a diffusion framework and under both discrete and continuous-time objectives [22, 23]. These developments demonstrate that masked prediction and diffusion can be combined in discrete settings, enabling efficient and flexible generation across different modalities.

2.1.3 Masked Image Generation

Masked image generation extends BERT-style [8] masked token prediction to images.

MaskGIT [9] generates ImageNet images in roughly 10 steps by iteratively predicting masked tokens using a categorical distribution. Sampling begins with all tokens masked and at each iteration, the model predicts probabilities and unmaskes the most confident tokens, gradually completing the image. During training, a random mask ratio r is sampled and tokens are masked according to a schedule γ_r , with the model optimising the cross-entropy loss

$$L(x) = \mathbb{E}_{r \sim U[0,1]} \mathbb{E}_{q_{\gamma_r}(x^M|x)} \left[\sum_{i: x_i = [M]} -\log p_{\theta}(x_i|x^M) \right].$$

Although MaskGIT operates over multiple steps, it differs from diffusion models as it does not explicitly model per-token noise or follow a Markov forward-backward process.

Extensions of masked generation have been proposed to operate in a continuous-valued space. MAR [24] propose an autoregressive image generation framework that eliminates the need for discrete vector-quantised tokenizers by representing each token as a continuous embedding. An autoregressive model predicts a conditioning vector z for each token, which parametrises a small per-token denoising diffusion network to model $p(x|z)$. This *Diffusion Loss* formulation allows joint training of the autoregressive model and denoiser, enabling sampling of continuous tokens through reverse diffusion and generalising both autoregressive and masked generation approaches, while achieving high-fidelity image synthesis.

2.1.4 Masked Diffusion

Diffusion models are a class of generative models that learn to generate data by reversing a gradual noising process, starting from a standard Gaussian prior and iteratively denoising towards a target image. The idea of using diffusion processes for generative modelling was first introduced by Sohl-Dickstein et al. [25], who proposed a framework in which data is gradually corrupted with Gaussian noise through a fixed forward process, and a model is trained to reverse this process to generate realistic samples. Building on this foundation, Ho et al. [?] developed Denoising Diffusion Probabilistic Models (DDPMs), which simplified the reverse process by directly predicting the noise at each step and enabled high-quality image generation in practice.

In continuous-time diffusion models the forward noising process is formulated as a Markov chain $q(\mathbf{x}_t | \mathbf{x}_{t-1})$, where each step depends only on the previous state, and the reverse denoising process $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ is trained to invert this chain. The Markovian structure provides well-defined likelihoods and smooth gradients for stable optimisation, and supports advanced sampling methods, including stochastic ancestral sampling for diverse outputs and deterministic DDIM solvers for consistent synthesis. These properties have made diffusion models the standard for image generation tasks, achieving high-fidelity results [10, 18, 11]. Common practical enhancements include classifier-free guidance, adaptive noise schedules, and multi-scale conditioning, which improve sample quality and controllability.

Masked generative models (MGMs) operate in discrete latent spaces, predicting only a subset of masked tokens at each step. This selective prediction makes them inherently sparse and allows computational resources to focus on uncertain regions, improving efficiency [26, 27, 28]. Parallel decoding further accelerates generation compared to sequential autoregressive models. As a result, MGMs can synthesise images in as few as 16 steps [27], considerably faster than conventional diffusion models that typically require hundreds of iterations.

2.2 Mathematical Framework of Discrete Diffusion Models

Notation.

Let tokens from a vocabulary of size V be represented as one-hot vectors:

$$\mathcal{V} \triangleq \left\{ x \in \{0, 1\}^V : \sum_{i=1}^V x_i = 1 \right\}. \quad (2.2)$$

We denote the categorical distribution by $\text{Cat}(\cdot; \pi)$ with $\pi \in \Delta^V$, the V -simplex. The K -th category is reserved for the mask token, represented by $m \in \mathcal{V}$. We also use $\mathbf{1}$ for the all-ones vector, $\langle a, b \rangle$ for dot products, and $a \odot b$ for elementwise multiplication.

The Forward Diffusion Process:

Consider the forward process for a single token. The forward process is modelled as a Markov chain of discrete random variables $\mathbf{x}_t$, indexed by a time $t \in [0, 1]$.



Figure 2.1: The Markov process visualised for 1 state from time step s to t .

The interval $[0, 1]$ is partitioned into T equal segments, and define $s(i) = \frac{i-1}{T}$ and $t(i) = \frac{i}{T}$. The transition from time $s(i)$ to $t(i)$ is described by a $(m+1) \times (m+1)$ transition matrix:

$$Q_i = (1 - \beta_i)I + \beta_i \mathbf{1e}_m^\top, \quad (2.3)$$

where I is the identity matrix of size $m+1$, $\mathbf{1}$ is an all-ones vector of length $m+1$, and $\mathbf{e}_m$ is the one-hot vector representing the mask token [29]. The entry $[Q_i]_{jk}$ gives the probability of transitioning from state j to state k :

$$[Q_i]_{jk} = q(\mathbf{x}_{t(i)} = k \mid \mathbf{x}_{s(i)} = j) = (1 - \beta_i)\delta_{jk} + \beta_i\delta_{km}. \quad (2.4)$$

At each step i , an unmasked token either remains unchanged with probability $1 - \beta_i$ or transitions to the absorbing mask state m with probability β_i . This can be seen as a Bernoulli trial applied independently to each token: survival occurs with probability $1 - \beta_i$, masking with probability β_i . The cumulative survival probability after i steps is

$$\alpha_i \triangleq \prod_{j=1}^i (1 - \beta_j), \quad (2.5)$$

which represents the probability that a token has not yet been masked. For a sequence of length L , the number of surviving unmasked tokens after i steps follows a binomial distribution with parameters L and α_i .

The marginal distribution of $\mathbf{x}_{t(i)}$ given the initial state $\mathbf{x}_0$ can be expressed as

$$q(\mathbf{x}_{t(i)} \mid \mathbf{x}_0) = \text{Cat}(\mathbf{x}_{t(i)}; \overline{Q}_i^\top \mathbf{x}_0) = \mathbf{x}_0^\top \overline{Q}_i \mathbf{x}_{t(i)}, \quad (2.6)$$

where $\text{Cat}(\mathbf{x}; \mathbf{p})$ denotes a categorical distribution with probability vector $\mathbf{p}$. The cumulative transition matrix up to step i is defined as

$$\overline{Q}_i \triangleq \prod_{j=1}^i Q_j = \alpha_i I + (1 - \alpha_i) \mathbf{1e}_m^\top. \quad (2.7)$$

For sufficiently large T , $\alpha_T \rightarrow 0$, causing $q(\mathbf{x}_1 \mid \mathbf{x}_0)$ to concentrate as a delta distribution on the mask state. Thus, as $T \rightarrow \infty$, the forward process converges to the fully masked distribution.

Continuous-time Limit:

Shi et al. [23] extends the discrete-time forward process to continuous time by taking the limit as $T \rightarrow \infty$. Define a continuous rate function $\beta(t)$ such that $\beta_i = \frac{\beta(t(i))}{T}$. Then, the cumulative transition matrix converges to:

$$\overline{Q}(t) \triangleq \lim_{T \rightarrow \infty} \overline{Q}_i = \alpha_t I + (1 - \alpha_t) \mathbf{1e}_m^\top, \quad (2.8)$$

where

$$\alpha_t \triangleq \exp \left(- \int_0^t \beta(s) ds \right). \quad (2.9)$$

The marginal distribution at time t given $\mathbf{x}_0$ is

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \text{Cat}(\mathbf{x}_t; \overline{Q}(t)^\top \mathbf{x}_0). \quad (2.10)$$

For any $0 \leq s < t \leq 1$, the transition kernel $q(\mathbf{x}_t \mid \mathbf{x}_s)$ compatible with these marginals is given by:

$$q(\mathbf{x}_t \mid \mathbf{x}_s) = \text{Cat}(\mathbf{x}_t; \overline{Q}(s, t)^\top \mathbf{x}_s), \quad (2.11)$$

where

$$\overline{Q}(s, t) \triangleq \overline{Q}(s)^{-1} \overline{Q}(t) = \frac{\alpha_t}{\alpha_s} I + \left(1 - \frac{\alpha_t}{\alpha_s} \right) \mathbf{1} \mathbf{e}_m^\top. \quad (2.12)$$

The explicit form of the transition matrix between arbitrary times s and t was not originally derived in Austin et al. [29] but later appeared in Zhao et al. [30].

Time Reversal Process Given $\mathbf{x}_0$:

The analytical property of the forward process means we can compute the time-reversed conditional distribution $q(\mathbf{x}_s \mid \mathbf{x}_t, \mathbf{x}_0)$ for $s \leq t$. This is derived in [23] and given by

$$q(\mathbf{x}_s \mid \mathbf{x}_t, \mathbf{x}_0) = \text{Cat}(\mathbf{x}_s; \overline{R}^{\mathbf{x}_0}(t, s)^\top \mathbf{x}_t), \quad (2.13)$$

where

$$\overline{R}^{\mathbf{x}_0}(t, s) = I + \frac{\alpha_s - \alpha_t}{1 - \alpha_t} \mathbf{e}_m (\mathbf{x}_0 - \mathbf{e}_m)^\top. \quad (2.14)$$

Here, $\overline{R}^{\mathbf{x}_0}(t, s) \in \mathbb{R}^{(m+1) \times (m+1)}$ is the transition matrix. This can be derived using Bayes' rule. Then in the reverse process conditioned on $\mathbf{x}_0$, if $\mathbf{x}_t$ is the mask token, with probability $\frac{\alpha_s - \alpha_t}{1 - \alpha_t}$ it transitions back to the original state $\mathbf{x}_0$ at time s , otherwise it remains masked. Once $\mathbf{x}_t$ is unmasked, it stays in that state for all earlier times until the end.

Derivation of the time-reversed posterior.

We start from the discrete-time conditional kernel (or its continuous-time analogue)

$$q(x_t \mid x_s) = \frac{\alpha_t}{\alpha_s} x_s + \left(1 - \frac{\alpha_t}{\alpha_s} \right) m, \quad 0 \leq s < t \leq 1, \quad (2.15)$$

and the marginal

$$q(x_t \mid x_0) = \alpha_t \delta_{x_t, x_0} + (1 - \alpha_t) \delta_{x_t, m}, \quad (2.16)$$

where δ denotes the Kronecker delta and m is the mask one-hot vector. Using Bayes' rule,

$$q(x_s \mid x_t, x_0) = \frac{q(x_t \mid x_s) q(x_s \mid x_0)}{q(x_t \mid x_0)}. \quad (2.17)$$

Case 1: $x_t \neq m$. If the observed state at time t is a non-mask token v (i.e. $x_t = v$ with $v \neq m$), then by construction of the absorbing process the only way to observe v at time t is that the token survived all intermediate masking events and therefore $x_s = v$. Formally, for any $y \neq v$ we have $q(x_t = v \mid x_s = y) = 0$, while $q(x_t = v \mid x_s = v) = \frac{\alpha_t}{\alpha_s} > 0$. Hence the numerator in Bayes' rule is zero for $x_s \neq v$ and nonzero for $x_s = v$, which yields the point mass such that

$$q(x_s \mid x_t = v, x_0) = \delta(x_s = v), \quad q(x_s \mid x_t \neq m, x_0) = \delta(x_s = x_t) \quad (2.18)$$

Case 2: $x_t = m$. We compute the posterior probabilities for the two possible antecedent states at time s : (i) the token was already unmasked and equal to x_0 at time s ; or (ii) the token was already masked at time s . By Bayes' rule, for the event $x_s = x_0$ we have

$$\begin{aligned} q(x_s = x_0 \mid x_t = m, x_0) &= \frac{q(x_t = m \mid x_s = x_0) q(x_s = x_0 \mid x_0)}{q(x_t = m \mid x_0)} \\ &= \frac{\left(1 - \frac{\alpha_t}{\alpha_s}\right) \alpha_s}{1 - \alpha_t} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}. \end{aligned} \quad (2.19)$$

Similarly, for the event $x_s = m$,

$$\begin{aligned} q(x_s = m \mid x_t = m, x_0) &= \frac{q(x_t = m \mid x_s = m) q(x_s = m \mid x_0)}{q(x_t = m \mid x_0)} \\ &= \frac{1 \cdot (1 - \alpha_s)}{1 - \alpha_t} = \frac{1 - \alpha_s}{1 - \alpha_t}. \end{aligned} \quad (2.20)$$

There are no other possible antecedent states because the chain only allows transitions from a non-mask token to itself or to the mask. Combining these two probabilities gives the categorical posterior as in Eq. (2.17).

Model and Objective:

In generation we do not have the ground truth $\mathbf{x}_0$, so we use a neural network f_θ to produce a probability distribution over possible tokens. This distribution, denoted $\mu_\theta(\mathbf{x}_t, t) \in \Delta^{m+1}$, is obtained by applying a softmax to the network output. The probability of mask token in the original is set to zero by setting its logit to $-\infty$:

$$\mu_\theta(\mathbf{x}_t, t) = \begin{cases} \text{softmax}(f_\theta(\mathbf{x}_t, t)) & \text{if } \mathbf{x}_t = [\text{MASK}], \\ \mathbf{x}_t & \text{otherwise.} \end{cases} \quad (2.21)$$

We define the reverse model as

$$p_\theta(\mathbf{x}_s \mid \mathbf{x}_t) \triangleq q(\mathbf{x}_s \mid \mathbf{x}_t, \mu_\theta(\mathbf{x}_t, t)), \quad (2.22)$$

where q is the analytic time-reversal of the forward process, as in Eq. (2.14). This setup is known as *mean parametrisation*, as the model predicts the expected original token. The network can be simplified to ignore time t , as shown in [31], since the conditional $p(\mathbf{x}_0 \mid \mathbf{x}_t = \mathbf{x})$ remains the same across t . Their GitHub implementation demonstrates that a variety of architectures can be used for prediction, including ViT, Sharded Transformer, standard Transformer, U-Net, and UViT.

Loss For Masked Discrete Diffusion Models

In discrete-time masked diffusion, the generative model approximately reverses the forward noising process via a learned reverse model $p_\theta(x_s \mid x_t)$ (Eq. (2.22)). The ELBO decomposes the log-likelihood of the data x_0 into three main components:

$$\log p_\theta(x_0) \geq \underbrace{\mathbb{E}_{q(x_T \mid x_0)} [\log p_\theta(x_0 \mid x_T)]}_{\text{Reconstruction}} - \underbrace{\text{KL}(q(x_1 \mid x_0) \parallel p(x_1))}_{\text{Prior matching}} - \underbrace{L_T}_{\text{Stepwise diffusion}}, \quad (2.23)$$

where

$$L_T = \sum_{t=2}^T \mathbb{E}_{q(x_t \mid x_0)} [\text{KL}(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t))]. \quad (2.24)$$

Prior Loss At maximum noise ($t = 1$), all tokens are masked, so the forward distribution simplifies to (Eq. (2.5) and Eq. (2.6)):

$$q(x_1 | x_0) \approx \delta_{x_1, m}.$$

Choosing the model prior to match this, $p(x_1) \approx \delta_{x_1, m}$, results in

$$\text{KL}(q(x_1 | x_0) \| p(x_1)) = 0.$$

Hence, the prior term contributes nothing to the gradient, confirming the theoretical justification for omitting this term in practical training.

Reconstruction Loss The reconstruction term

$$\mathbb{E}_{q(x_T | x_0)}[\log p_\theta(x_0 | x_T)] \quad (2.25)$$

becomes trivial as $x_T \rightarrow x_0$ at the final diffusion step. For discrete spaces, it reduces to a constant in the **unconditional** case:

$$L_{\text{recon}}^{\text{uncond}} = \prod_{\text{dims}} (1 - \alpha_{t \rightarrow 0}) \log |V|, \quad (2.26)$$

which provides negligible learning signal but confirms theoretical consistency with the ELBO framework [22].

For **class-conditional** generation, the reconstruction term becomes:

$$L_{\text{recon}}^{\text{cond}} = \prod_{\text{dims}} (1 - \alpha_{t \rightarrow 0}) H[x_0 | y], \quad (2.27)$$

where $H[x_0 | y]$ is the conditional entropy of the data given the class label y . Conditioning reduces the entropy relative to the uniform distribution, but this term is still **independent of the model parameters** and provides no meaningful gradient. Even with conditioning, the reconstruction loss remains a **bounded constant**, which is why many masked diffusion models [22] ignore it in practice and focus on the stepwise diffusion loss L_T for training.

Stepwise Diffusion Loss The primary learning signal arises from L_T (Eq. (2.24)), which for discrete diffusion simplifies to a weighted cross-entropy over masked positions:

$$\text{KL}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t)) = -\frac{\alpha_{t-1} - \alpha_t}{1 - \alpha_t} \delta_{x_t, m} x_0^\top \log \mu_\theta(x_t, t), \quad (2.28)$$

where $\mu_\theta(x_t, t)$ are the predicted token probabilities (Eq. (2.21)), and only positions where $x_t = m$ contribute. The weighting $(\alpha_{t-1} - \alpha_t)/(1 - \alpha_t)$ reflects the probability of unmasking at each step (Eq. (2.14)).

Connection to Continuous Diffusion

Taking the continuous-time limit $T \rightarrow \infty$, the discrete sum becomes a Riemann integral:

$$L_\infty = \int_0^1 \frac{d\alpha_t}{dt} \frac{1}{1 - \alpha_t} \mathbb{E}_{q(x_t | x_0)} \left[\sum_{n: x_t^{(n)} = m} x_0^{(n)\top} \log \tilde{\mu}_\theta^{(n)}(x_t, t) \right] dt. \quad (2.29)$$

Reparameterising with the log-SNR $\lambda_t = \log(\alpha_t/(1 - \alpha_t))$ clarifies how masking probabilities weight the loss across diffusion time, paralleling the continuous MSE loss in standard DDPMs [10].

Practical Masked Cross-Entropy Objective

The resulting practical loss is

$$L = \mathbb{E}_{t, x_0, x_t \sim q(x_t | x_0)} \left[\sum_{n: x_t^{(n)} = m} \text{CrossEntropy}(\mu_\theta^{(n)}(x_t, t), x_0^{(n)}) \right], \quad (2.30)$$

which is numerically stable, computationally efficient, and theoretically grounded. It avoids the complexity and potential instability of full D3PM KL-based losses [22].

Relation to Masked Language Modeling

This loss can be interpreted as a time-dependent, weighted masked language modeling objective:

$$L_{\text{diffusion}} = \mathbb{E}_t [w(t) L_{\text{MLM}}(1 - \alpha_t)], \quad w(t) = \frac{\alpha'_t}{1 - \alpha_t}, \quad (2.31)$$

The curriculum of masking rates naturally encourages hierarchical, coarse-to-fine generation, promoting both local accuracy and global semantic coherence [22].

The simplified masked cross-entropy loss captures the essential dynamics of discrete diffusion while remaining computationally efficient and numerically stable. By unifying discrete diffusion and BERT-style masked language modelling in a principled framework, it highlights that the choice of masking schedule is a more critical design decision than complex KL formulations.

2.3 Model Architectures

In both autoregressive (AR) and masked-diffusion formulations, the central task is to predict discrete tokens using a neural network. Within diffusion models, a widely adopted backbone is the U-Net, an encoder-decoder convolutional neural network with skip connections and ResNet-style residual blocks. U-Nets were first incorporated into diffusion models in DDPMs, enhanced with spatial self-attention at lower resolutions and conditioning on timestep embeddings. In class-conditional variants, label embeddings are injected through adaptive normalisation layers within residual blocks [10, 15]. This design has proven highly effective and has since become the standard architecture across modalities. In image generation, U-Nets serve as the core architecture for unconditional and class-conditional generation. Successive models progressively refine images at higher resolutions [?]. The model remains a popular choice due to its strong inductive biases toward preserving spatial structure via skip connections, efficient multi-scale feature learning.

Recent work has proposed Diffusion Transformers (DiTs) as a unified backbone for diffusion models [32]. DiTs follow the design of Vision Transformers (ViTs) [14], operating on sequences of latent patches rather than spatial feature maps, and were shown to match or surpass U-Nets when trained within latent diffusion frameworks [11]. Whereas U-Nets use convolutional structures and skip connections to encode locality and multi-scale interactions, DiTs employ only self-attention and feedforward layers, eliminating convolutional biases and enabling more scalable architectures. Scaling experiments indicate that generation quality improves nearly linearly with compute, with large DiTs achieving top performance on class-conditional ImageNet [32]. Both architectures have been applied to continuous and discrete image diffusion.

Autoregressive image models commonly employ decoder-only Transformers with causal self-attention, learned or sinusoidal positional encodings, and class conditioning. Representative architectures include the Image Transformer as well as autoregressive decoders trained on VQ-VAE [3] or VQGAN [16] latents.

2.4 Conditioning in Models

Conditional generation is central to modern generative models, requiring mechanisms to incorporate class labels, diffusion time-steps, and contextual information in order to guide and control the synthesis process. Early work demonstrated that injecting class information significantly boosts the fidelity and diversity of generated images [15], while later extensions explored conditioning on text [33, 11], or audio [34]. Both convolutional U-Nets and transformer-based Diffusion Transformers (DiTs) incorporate mechanisms to inject conditional signals—most commonly timestep and class information—into the backbone architecture.

In Context Conditioning In-context conditional generation refers to conditioning generative models directly on auxiliary information provided as part of the input sequence or feature representation, rather than through architectural modifications. Formally, given a sequence of data tokens $x = (x_1, \dots, x_n)$ and conditioning tokens $c = (c_1, \dots, c_m)$, the model input is constructed as the concatenated sequence

$$z = [c_1, \dots, c_m, x_1, \dots, x_n],$$

which is processed by a transformer with self-attention layers, enabling joint attention across conditioning and data tokens. This formulation allows the model to implicitly learn the conditional distribution $p(x | c)$ using the attention, without requiring explicit conditioning modules. This approach is most commonly seen in large-scale language models, where conditioning is provided through text prompts [35, 4], and has been extended to diffusion transformers by embedding class or text tokens into the input sequence [33, ?].

Cross-attention conditioning. Cross-attention allows diffusion models to incorporate conditioning by attending to them in a cross attention block. Conditioning is introduced by providing an additional set of key-value pairs derived from context tokens $c = (c_1, \dots, c_m)$, while queries are computed from the primary data sequence $x = (x_1, \dots, x_n)$. At each attention layer, the mechanism is defined as

$$\text{CrossAttn}(Q_x, K_c, V_c) = \text{softmax}\left(\frac{Q_x K_c^\top}{\sqrt{d}}\right) V_c,$$

where $Q_x = xW_Q$, $K_c = cW_K$, and $V_c = cW_V$. Cross-attention keeps the data tokens and conditioning tokens distinct. Isolating conditioning features from primary data gives the model greater control over how context affects generation.

In diffusion models, cross-attention has been particularly effective for text-to-image generation, where text embeddings produced by pretrained language models guide the denoising process [11]. The mechanism facilitates modelling of the conditional distribution $p(x | c)$ by dynamically aligning data queries with contextual features.

In Diffusion Transformers (DiT), cross-attention is implemented by concatenating timestep and class embeddings into a short sequence distinct from the image tokens. Each block includes a multi-head cross-attention layer after self-attention, following Vaswani et al. [36] and the conditioning strategy of Latent Diffusion Models [32, 11]. While adding roughly 15% Gflops over self-attention, cross-attention provides flexibility for structured or variable-length conditioning and proves useful in multimodal settings, enabling alignment between heterogeneous representational domains.

Adaptive Layer Normalization (AdaLN and AdaLN-0). AdaLN injects conditioning information directly into the normalisation layers of the transformer. Instead of using

fixed scale and shift parameters, AdaLN makes them dependent on a conditioning vector c :

$$\gamma' = \gamma + f_\gamma(c), \quad \beta' = \beta + f_\beta(c),$$

where f_γ, f_β are small neural networks (often linear layers) mapping c into the same dimensionality as the hidden representation [32]. The transformed output is then

$$h' = \frac{h - \mu(h)}{\sigma(h)} \odot \gamma' + \beta'.$$

AdaLN provides a mechanism to bias the model’s computations at every layer, so that conditioning inputs continuously adjust how representations are scaled and shifted throughout the model’s layers.

AdaLN-Zero (AdaLN-0). AdaLN-Zero is a variant of Adaptive Layer Normalisation designed for transformer-based diffusion models [32]. In AdaLN-0, the base learnable parameters γ and β are fixed to 1 and 0, respectively, and the conditioning projections $f_\gamma(c), f_\beta(c)$ are initially zero. This ensures that at the start of training the network behaves as an unconditioned identity. Additionally, AdaLN-0 introduces a learnable, conditioning-dependent scaling factor $\alpha(c)$ applied to the residual branch,

$$y = h + \alpha(c) \cdot F(\text{AdaLN}(h)),$$

where F denotes the attention or feedforward block. This residual scaling allows the conditioning signal to be incorporated gradually, improving training stability while preserving the ability to steer activations through attention and linear layers. This design is computationally efficient, and applies a uniform function to all tokens, maintaining consistency across the sequence and improve training stability, while preserving the network’s capacity to incorporate class and timestep conditioning. The authors of Scalable Diffusion Models with Transformers [32] evaluated four conditioning strategies—including cross-attention, AdaLN, and AdaLN-Zero—and found that AdaLN-Zero achieved the lowest FID but and offered the highest compute efficiency. As such, it has become the conditioning strategy of choice in recent masked discrete diffusion models [22, 23]. This parallels the role of adaptive normalisation (AdaGN, AdaBN) in U-Net-based backbones, where class or timestep embeddings modulate the feature maps at each layer to condition the generation. In U-Nets, the convolutional structure and hierarchical feature maps make per-channel modulation effective, whereas in transformers, AdaLN-0 adjusts the scale and shift of activations across attention and linear layers, which is better suited to the token-wise, sequence-based computation of DiTs.

2.5 Conditional Generation

Classifier-Free Guidance Class conditioning allows the model to incorporate class information into the input, guiding it to generate samples corresponding to a specific class. Conditional diffusion models sometimes under-utilise the conditioning input y during generation as the model can generate data without the condition, leading to weaker conditional fidelity. Classifier-Free Guidance (CFG) [37] addresses this by interpolating between conditional and unconditional noise predictions. The conditioning essentially learns a direction in which to remove the noise dependent on class.

During training, a fraction of conditioning signals y are randomly dropped, so the network learns both the conditional distributions $\hat{\epsilon}_\theta(x_t, t, y)$ and $\hat{\epsilon}_\theta(x_t, t, \emptyset)$. At inference, the guided noise estimate is computed as:

$$\hat{\epsilon}_{\text{CFG}} = \hat{\epsilon}_{\text{uncond}} + w \cdot (\hat{\epsilon}_{\text{cond}} - \hat{\epsilon}_{\text{uncond}})$$

where the guidance scale $w > 1$ controls the strength of conditioning [38].

This approach can be interpreted as approximating the gradient of the conditional log-probability:

$$\nabla_{x_t} \log p(y | x_t) \approx \hat{s}_\theta(x_t, y) - \hat{s}_\theta(x_t)$$

CFG directly modifies the score estimates at each timestep. These methods are often combined, as seen in models like GLIDE [39] and Stable Diffusion [11], to achieve improved conditional generation quality. In practice, while training, a fraction of class labels are randomly dropped, encouraging the network to learn both conditional and unconditional distributions. At inference, the model predicts both conditional and unconditional logits for each token, which are then combined using a guidance weight w to adjust the final logits. Sampling is performed from these modified logits, allowing a trade-off between adherence to the conditioning signal and sample diversity. This method has become a standard approach for incorporating class or context information in diffusion models without requiring explicit external classifiers.

2.6 Noise Schedule

In diffusion generative models, the noise schedule is a central design component. It controls how corruption is applied in the forward process and therefore the difficulty of the reverse denoising task. In the Gaussian setting, the schedule is defined by variance parameters $\{\beta_t\}_{t=1}^T$, or equivalently by the cumulative product $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$, which reflects the signal to noise ratio (SNR) at time t [10]. In discrete masked diffusion, the same principle applies. Tokens are replaced with a mask symbol with probability β_t and retained with probability $\alpha_t = 1 - \beta_t$ [29]. Thus the cumulative keep rate $\bar{\alpha}_t$ plays an analogous role, and schedule design considerations translate directly from continuous to discrete cases. A well-designed noise schedule gradually degrades the signal to prevent premature collapse, provides sufficient coverage of intermediate noise levels to guide the reverse process in restoring the input, and ensures that the signal-to-noise ratio (SNR) vanishes at high timesteps to avoid residual signal leakage as $t \rightarrow T$ [10, 40]. Different schedules vary in how they distribute corruption, effecting both the forward degradation and the gradients available to the denoiser.

Linear Schedule

The *linear schedule*, introduced in DDPMs [10], increases β_t uniformly:

$$\beta_t = \beta_1 + \frac{t-1}{T-1}(\beta_T - \beta_1). \quad (2.32)$$

This strategy has been shown to decay the signal-to-noise ratio too rapidly at early steps, leading to poor utilisation of the middle range of timesteps. This limitation carries over directly to masked diffusion, where a linear increase in masking probability results in an excess of tokens being replaced too early.

Cosine Schedule

Cosine-based schedules are widely used in masked diffusion. One formulation defines the cumulative keep probability as

$$\alpha(t) = 1 - \cos\left(\frac{\pi}{2}(1-t)\right), \quad t \in [0, 1], \quad (2.33)$$

This schedule slows down masking at early timesteps, preserving more signal when the denoiser is least informed and reducing the risk of conflicting tokens being masked or unmasked simultaneously [9].

Squared-Cosine Schedule

Nichol and Dhariwal [41] proposed the *squared-cosine schedule*, which parameterises $\bar{\alpha}_t$ directly:

$$\bar{\alpha}_t = \cos^2 \left(\frac{t/T + \epsilon}{1 + \epsilon} \cdot \frac{\pi}{2} \right), \quad (2.34)$$

where a small $\epsilon > 0$ prevents $\bar{\alpha}_1$ from starting exactly at 1, ensuring a nonzero noise. By following the squared cosine curve, this schedule retains most of the signal near $t = 0$ and $t = T$. Instead of defining β_t directly, this method parameterises the cumulative retain rate $\bar{\alpha}_t$ as a shifted cosine function and maintains higher signal retention near the boundaries while concentrating denoising difficulty around intermediate timesteps. When applied in masked diffusion, this results in more balanced gradients, since tokens are masked at a rate that aligns with the denoiser’s capacity. Empirically, the cosine schedule improves sample quality, stabilises training, and accelerates convergence compared to linear schedules [41, 40, 29].

Polynomial Schedule

The *polynomial schedule* generalises the linear case by raising the timestep ratio to a power γ [40]:

$$\beta_t = \beta_{\min} + \left(\frac{t}{T} \right)^\gamma (\beta_{\max} - \beta_{\min}). \quad (2.35)$$

where $\gamma > 0$ adjusts the curvature. $\gamma = 1$ recovers the linear schedule, $\gamma > 1$ delays large noise until later steps, and $\gamma < 1$ injects stronger noise earlier. Using a polynomial schedule for $\bar{\alpha}_t$ ensures that most tokens are unmasked in the early steps, providing the denoiser with substantial signal for learning, while gradually increasing the masking probability so that all tokens are fully masked by the final step. This balances early signal preservation with convergence to a fully masked state, facilitating effective learning across timesteps, though they may require careful tuning compared to other schedules.

Sigmoid Schedule

The *sigmoid schedule* introduces a smooth transition:

$$\bar{\alpha}_t = -\frac{\sigma\left(\frac{t-s}{\tau}\right) - \sigma\left(\frac{-s}{\tau}\right)}{\sigma\left(\frac{1}{\tau}\right) - \sigma\left(\frac{-s}{\tau}\right)}, \quad (2.36)$$

where s shifts the curve and τ controls steepness. Compared with cosine, the sigmoid decays more gradually early on and flattens at later steps, which helps avoid abrupt changes in difficulty and improves robustness, especially in high-resolution settings. Empirical results suggest it yields more stable training while maintaining competitive sample quality, though differences in performance relative to cosine diminish at inference time. [42, 40].

The noise schedule shapes training by determining how many tokens are corrupted at each step and therefore the strength of the gradient signal. Linear schedules often overemphasise extremes, while cosine, polynomial, and sigmoid variants distribute corruption more evenly, improving stability and sample quality. In Gaussian diffusion, the ELBO reduces to a denoising loss [10], while in masked diffusion the absorbing prior causes the

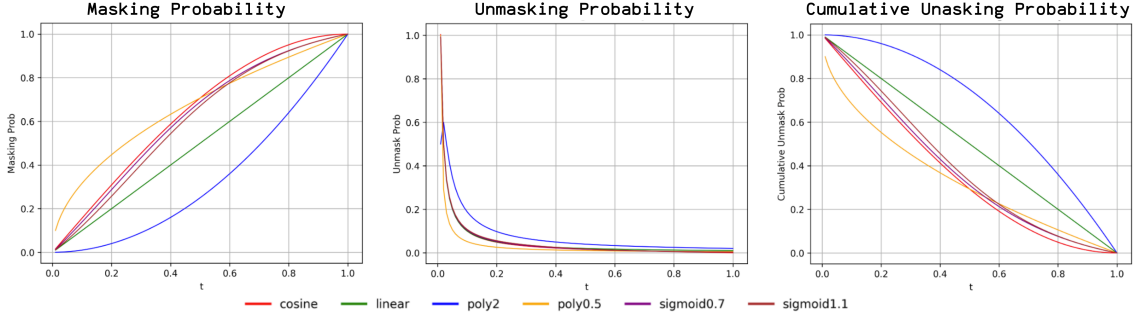


Figure 2.2: (Left) Masking probability, (Middle) Unmasking probability, (Right) Cumulative masking probability across different noise schedules: linear, cosine, polynomial (power 0.5), polynomial (power 2), sigmoid with $\tau = 0.7$, and sigmoid with $\tau = 1.1$.

objective to collapse to a masked cross-entropy [22, 43]. Thus, schedule design directly determines gradient allocation across timesteps.

Recent studies highlight both empirical and theoretical aspects. Shi *et al.* [23] showed that linear and polynomial schedules perform competitively in practice, while Zhang [44] justified cosine schedules as near-optimal under Fisher–Rao geometry. Other approaches, such as DUS [45] and MDPO [46], focus on inference optimisation rather than altering corruption directly. Overall, the schedule remains a critical factor in both continuous and discrete diffusion, with cosine and polynomial variants widely adopted as practical defaults.

2.7 Self-Attention Masking Mechanics

Self-attention computes queries $Q = XW_Q$, keys $K = XW_K$, and values $V = XW_V$, with attention weights $A = \text{softmax}(QK^T/\sqrt{d_k})$ and output $Z = AV$, where $X \in \mathbb{R}^{n \times d}$ is the input sequence, d_k the key dimension, and n the sequence length. Masks are applied before the softmax by setting masked positions to $-\infty$, so $\exp(-\infty) = 0$ and these entries receive zero attention: $\tilde{A}_{ij} = Q_i K_j^T / \sqrt{d_k}$ if unmasked, $-\infty$ if masked, with $A_{ij} = \exp(\tilde{A}_{ij}) / \sum_k \exp(\tilde{A}_{ik})$.

Row masking occurs when all positions in row i are masked. Implementations either set the row to zeros, giving $Z_i = 0$ and $Y_i = X_i$, or assign uniform weights $A_{ij} = 1/n$, averaging the value vectors. Column masking prevents attention to position j in the sequence, with remaining weights renormalized over unmasked entries. Residual connections ensure gradient flow even when attention outputs are zero, and numerically stable softmax subtracts the row maximum in the exponent to avoid indeterminate forms. Masking allows control over the flow of information, ensuring that the model only has access to the context that will be available at inference.

Causal Masking Autoregressive models are trained using a single forward pass per sequence, with a causal mask applied to enforce a left to right generation order. This corresponds to teacher forcing [?], where the model only attends to previous or current tokens, allowing fast and stable training. Formally, the masked attention is defined as:

$$\tilde{A}_{ij} = \begin{cases} Q_i K_j^T / \sqrt{d_k} & j \leq i \\ -\infty & j > i \end{cases}.$$

Here, position i can only attend to tokens up to i , preserving causal structure and preventing mathematical instabilities.

By restricting attention to past tokens, causal masking ensures that the model learns a valid autoregressive factorisation of the joint distribution, simplifying optimisation since

each token prediction only depends on known context rather than the entire sequence. This allows training with a single forward and backward pass per sequence, rather than requiring iterative or sequential updates. In practice, this strategy is widely used in language models such as GPT [35, 4] and in image generation models that operate autoregressively on pixels or discrete latent tokens, including the Image Transformer [14], VQ-VAE decoders [?], and VQGAN autoregressive models [7]. In all cases, causal masking enables stable, parallelisable training while preserving the sequential generation constraints inherent to autoregressive modelling.

Chapter 3

VAR Architecture and Baseline

In this project, I develop a method for hierarchical generation of discrete data, where the model initially produces coarse, low-resolution features and then progressively refines them to capture finer details. The approach combines autoregressive modelling between scales with diffusion-based refinement within each scale, enabling the model to represent both global structure and local detail efficiently. The generative model over discrete data is designed to maintain a bidirectional understanding of the full feature map, striking a balance between high fidelity and computational tractability. Drawing inspiration from the next-resolution prediction mechanism in Visual AutoRegressive (VAR) models [13], I implement an analogous strategy incorporating diffusion.

The VAR framework [13] reformulates autoregressive modelling from next-token prediction to next-scale prediction. Whereas traditional autoregressive methods generate each image patch sequentially in raster order, producing unidirectional dependencies along the sequence, VAR predicts complete token maps at successive spatial resolutions, with each resolution conditioned on the entire representation at the previous scale. This shifts the modelling objective from capturing purely local dependencies to modelling global, scale-to-scale structure, providing a natural foundation for hierarchical generation in discrete domains.

VAR address several fundamental limitations of conventional autoregressive image models. VAR overcomes the unidirectional limitation of standard autoregressive models by predicting entire token maps at successive spatial resolutions instead of one token at a time. In quantised auto-encoders (VQ-VAEs), feature vectors are naturally interdependent, but flattening them into a linear token sequence imposes a unidirectional dependency, where each token can only depend on its predecessors, contradicting the true feature correlations. VARs also improve zero-shot generalisation, allowing inference across the image in a bidirectional manner, unlike traditional autoregressive models which cannot predict, for example, the top of an image given the bottom. Furthermore, VAR preserve spatial locality by maintaining correlations between neighbouring tokens that are lost during flattening. They also improve computational efficiency by generating entire resolution maps simultaneously, rather than sequentially producing $n \times n$ tokens, which would require $O(n^2)$ autoregressive steps and $O(n^6)$ operations. These advantages make VAR a natural framework for hierarchical, coarse-to-fine image generation [13].

3.1 Tokenisation

3.1.1 Multi-Scale Tokenisation

The formulation relies on a VQ-VAE, which quantises feature maps f

$$f \in \mathbb{R}^{h \times w \times C} \quad p(r_1, r_2, \dots, r_K) = \prod_{k=1}^K p(r_k \mid r_1, r_2, \dots, r_{k-1}).$$

into K token maps $(r_1, r_2, \dots, r_K)$. Each r_k corresponds to tokens at resolution $h_k \times w_k$, with resolutions increasing until r_K matches the original spatial dimensions H, W . Each $r_k \in [V]^{h_k \times w_k}$ contains $h_k \times w_k$ tokens from a vocabulary of size V . The sequence $(r_1, \dots, r_{k-1})$ serves as conditioning context for predicting r_k . The token maps are accumulated across scales, meaning r_k aims to predict the difference between the true image and the cumulative predictions $r_{\leq k}$. This satisfies the autoregressive condition that r_k is only dependent on $r_{\leq k}$, which is broken in most other AR image generation models. This design preserves spatial locality as no flattening operation is applied to token maps, and tokens within each r_k remain spatially correlated. As a result, the computational cost of generating an $n \times n$ latent image is reduced to $\mathcal{O}(n^4)$.

3.1.2 Multi-Scale VQ-VAE for VAR Learning

VAR employs a modified VQGAN [7] as its autoencoder, extending it to support multi-scale quantisation. In VQGAN, a 256×256 image is encoded into a 32×32 latent map with a codebook of 4096 entries. Training minimises a composite loss

$$\mathcal{L} = L_{\text{recon}} + L_{\text{latent}} + L_P + L_G + L_{\text{VQ}}, \quad (3.1)$$

where $L_{\text{recon}} = \|\text{im} - \hat{\text{im}}\|_2^2$ enforces pixel-level fidelity, $L_{\text{latent}} = \|f - \hat{f}\|_2^2$ aligns multi-scale features, L_P is a perceptual loss based on pretrained features [?, ?], and L_G is an adversarial loss from a GAN discriminator. The vector quantisation loss

$$L_{\text{VQ}} = \|\text{sg}[z_e(x)] - e\|_2^2 + \beta \|z_e(x) - \text{sg}[e]\|_2^2$$

penalises mismatch between encoder outputs $z_e(x)$ and codebook embeddings e , with sg the stop-gradient operator [7] and β a commitment weight [?]. which prevents gradients from flowing from the encoder into the codebook embeddings. Without this, the embeddings would continuously follow the encoder outputs, causing the latent space to collapse and reducing codebook diversity.

Multi-Scale Quantisation For VAR, the encoder output $f = E(\text{im})$ is processed hierarchically across K scales. Residual embeddings $\phi_k(z_k)$ are used to progressively encode information at each scale. Let f_k denote the feature map at scale k , and r_k the downsampled image at resolution (h_k, w_k) . The encoding at each scale proceeds as follows.

First, the feature map is interpolated to the target resolution and quantised to the nearest entries in the shared codebook Z :

$$z_k = \text{lookup}(Z, Q(\text{interpolate}(f_k, h_k, w_k))),$$

where z_k represents the discrete codebook embeddings corresponding to r_k .

Next, the residual embedding is computed as the difference between the current feature map and the information captured by the quantised embedding, and then subtracted from the feature map for the next scale:

$$f_{k+1} = f_k - \phi_k(z_k), \quad \phi_k(z_k) = f_k - \tilde{f}_k,$$

where $\tilde{f}_k$ is the reconstructed feature map at scale k derived from z_k . This residual subtraction allows each finer scale to focus exclusively on information not already captured by coarser scales. The model encodes only the remaining details at each scale, preventing redundancy and ensuring that the hierarchical representation accumulates information in a coarse-to-fine manner.

Reconstruction During decoding, each token map r_k is mapped back to its embedding z_k and interpolated to full resolution. Residuals are accumulated to reconstruct the feature map:

$$\hat{f} = \sum_{k=1}^K \phi_k(z_k), \quad \hat{\text{im}} = D(\hat{f}),$$

where D denotes the decoder. This residual multi-scale design preserves high-fidelity reconstruction and hierarchical structure suitable for VAR-style autoregressive generation [13].

Residual Embeddings In the multi-scale VQ-VAE used for VAR, the residual embedding $\phi_k(z_k)$ is a learned convolutional module applied to the feature map at each scale and is trained jointly with the encoder and decoder as part of the VQ-VAE / VAR objective. It plays a central role in the hierarchical encoding process. For each scale k , the encoder output $f_k \in \mathbb{R}^{B \times C \times H \times W}$ is interpolated and quantised to discrete codebook embeddings z_k , which capture the information at that resolution. The residual embedding $\phi_k(z_k)$, implemented as a small convolutional block in feature space, is then subtracted from the current feature map:

$$f_{k+1} = f_k - \phi_k(z_k).$$

This subtraction occurs in the encoder’s feature space (channels $\times$ height $\times$ width), not in image space. Intuitively, $\phi_k(z_k)$ represents the portion of the feature map that is already explained by the quantised tokens at scale k . By removing this component in feature space rather than in image space, the subsequent scale f_{k+1} focuses solely on the residual information that has not yet been captured, enabling each finer scale to encode details incrementally without redundancy. Operating in feature space avoids repeatedly decoding and re-encoding images, which is computationally expensive, and allows the model to refine representations directly on compact, high-dimensional feature maps. This approach ensures efficient, coarse-to-fine hierarchical representation and stabilises multi-scale reconstruction.

3.1.3 Transformer Architecture

The VAR framework employs a decoder-only Transformer similar to GPT-2. Multi-scale tokens are processed through standard self-attention mechanisms, with position embeddings adapted for each resolution scale and level embeddings distinguish between different scales. Conditional class information is introduced using Adaptive Layer Normalisation (AdaLN), which provides start tokens for initial generation and conditioning at later stages.

3.1.4 Input Representation and Processing

The VAR training process begins by preparing multi-scale visual tokens into a format suitable for the transformer. The model receives two primary inputs: class labels for conditional generation and teacher-forcing token sequences that provide ground-truth context during training.

The teacher-forcing input, denoted $x_BLCv_wo_first_1$, is constructed from the multi-scale VQ-VAE token maps. For each scale i , the corresponding token map is first embedded via the VQ-VAE codebook, then interpolated to the resolution of the next finer scale $i + 1$. These upsampled feature maps are progressively combined to provide context for generating

each finer scale. For the first resolution level, the sequence is instead initialised with class-conditioned start tokens. Formally, this process corresponds to the ‘idxBl_to_var_input’ function, which takes the list of ground-truth multi-scale token indices and produces a concatenated tensor of shape $B \times L - (first_l) \times C_{vae}$ representing the upsampled embeddings for all subsequent scales. Here, B is the batch size, L the total sequence length across scales, and C_{vae} the VQ-VAE embedding dimension.

Once constructed, the teacher-forcing tokens are projected to the transformer model dimension C via a learned linear layer, *word_embed*, allowing the discrete visual tokens to interact with the transformer’s continuous representation space. Positional embeddings *pos_1LC* encode spatial relationships both within and across scales, while level embeddings *lvl_embed* indicate the scale currently being processed. Additionally, start position embeddings *pos_start* provide initial spatial context for the first resolution level.

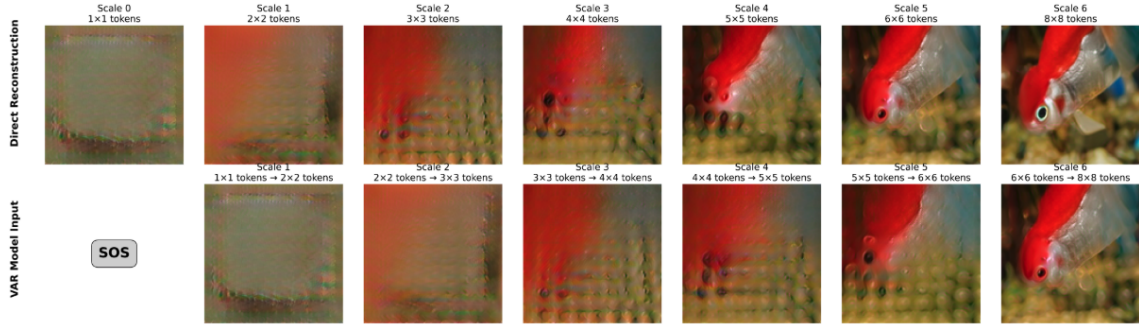


Figure 3.1: The top row shows the image quantized to a given scale, and the bottom row shows the corresponding model input for that scale.

As shown in Figure 3.1, the model input at scale $i + 1$ is obtained by upsampling the previous scale from pn_i^2 tokens to pn_{i+1}^2 tokens, while representing the same image. Here, pn denotes the number of patches at a given resolution i (note that in our terminology we refer to the squared number of patches, even though it is technically the root). Small differences arise due to processing by the quantizer and the addition of the residual embedding, which represents $\phi_k(z_k)$ via a convolutional network. This introduces slight reconstruction errors when representing the same image at different token resolutions. These discrepancies are more noticeable for some images than others.

3.1.5 Attention Masking Mechanisms

VAR enforces an autoregressive constraint across hierarchical scales while allowing full attention within each scale. A block-causal attention mask (*attn_bias_for_masking*) ensures tokens at scale k attend only to tokens from scales 1 through k , preventing information leakage. Mask values are 0 for allowed connections and $-\infty$ for prohibited ones, zeroing attention to future tokens. During inference, masking is removed, and key-value caching reuses generated scales to improve efficiency.

3.1.6 Token Prediction Objective

VAR predicts discrete visual tokens from a VQ-VAE codebook using a cross-entropy loss at each hierarchical scale [13]. The model generates logits over vocabulary size V , conditioned on all preceding scales. This hierarchical formulation decomposes the loss across scales, allowing evaluation of scale-specific prediction difficulty. To maintain gradient flow during progressive training, a zero-weighted sum of *word_embed* parameters is added to the loss, ensuring all model parameters remain connected in the computational graph.

Chapter 4

The Model

4.1 Model Input

4.1.1 Teacher Forcing Training

The Diffusion VAR framework requires adaptations to handle both masked and unmasked tokens during training. The model input is formed by concatenating the multi-scale discrete token maps with the corrupted ground-truth sequence. Concretely, the model receives

$$([s], r_1, r_2, \dots, r_{K-1}, n_0, n_1, \dots, n_K),$$

where n_i denotes the noised token map at scale i . The objective is to predict the clean sequence $(r_1, r_2, \dots, r_K)$ by progressively unmasking corrupted tokens.

The noised embeddings consist of a mixture of original ground-truth tokens and masked tokens, reflecting the forward diffusion process. Corruption is applied via the function $q(x_t)$, where, given ground-truth token sequences $x_0 \in \mathbb{Z}^{B \times L}$ and a masking probability p (determined by the sampled timestep t and the noise schedule), a subset of tokens is stochastically replaced with a dedicated mask token. For unmasked positions, $q(x_t)$ retains the original VAE embeddings, while masked positions are filled with a learned mask embedding, as introduced in the Diffusion VAR model.

This formulation enables the model to learn predictions under varying levels of corruption, mirroring the forward noising process in masked diffusion [?]. The timestep t is explicitly provided as a conditioning signal, ensuring the model is aware of the current noise level and can carry out accurate denoising.

4.1.2 Attention Mask

The hierarchical autoregressive constraint in DiffusionVAR is enforced through a structured attention mask, which simultaneously enables within-scale attention for masked inputs. Let T denote the total number of tokens across all scales, and let x_0 and x_t represent the ground-truth input and noised token embeddings, respectively. The attention mask is defined over the concatenated sequence $[x_0, x_t]$ of length $2T$.

To ensure that x_t depends on the input x_0 , we enforce teacher forcing. One approach considered was using a block diffusion mask similar to BD3LM [47], which consists of: (i) a block-causal mask for $x_0 \rightarrow x_0$, allowing each $x_0^{(i)}$ to attend only to preceding x_0 tokens; (ii) a block-diagonal mask for $x_t \rightarrow x_t$, allowing within-scale attention among noised tokens; and (iii) an offset block-causal mask for $x_0 \rightarrow x_t$, ensuring each $x_0^{(i)}$ can attend only to $x_t^{(j)}$ with $j > i$, preventing access to its own prediction. BD3LM conditions on both clean tokens x_0 and noised tokens x_t to enable effective denoising through contextual cross-attention, where the clean tokens x_0 serve as high-quality conditional context

for reconstructing corrupted tokens x_t . The clean tokens undergo self-attention updates through the block-causal mask to build contextualized representations that capture dependencies across previous blocks, making them more informative as conditional context for the denoising process. BD3LM implements diffusion over language while maintaining autoregressive dependencies between fixed size blocks, which allows generation of long or variable-length sequences. Although my setting differs, performing within-scale diffusion and autoregression across hierarchical image scales. Therefore, as each scale contains a different number of tokens, the blocks sizes must vary to match the number of tokens per scale.

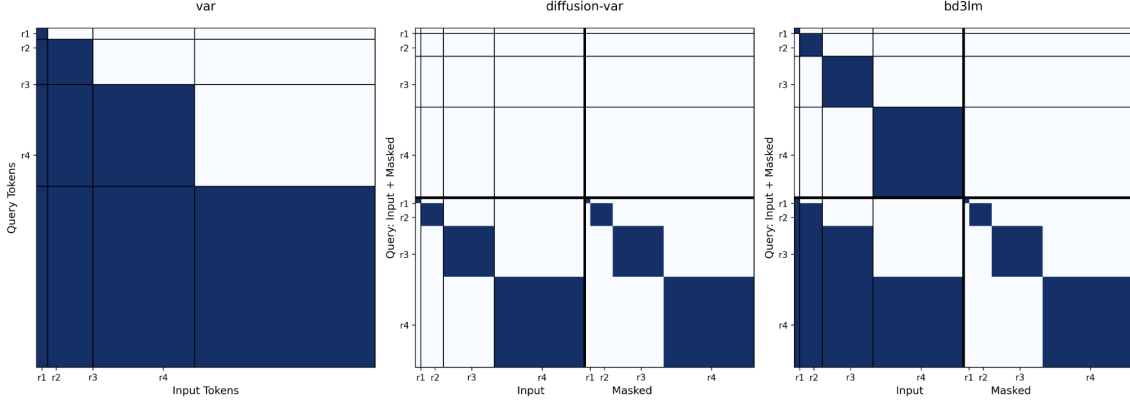


Figure 4.1: Comparison of masking strategies for VAR and hierarchical diffusion. Example using $patch_nums = (1, 2, 3, 4)$: VAR mask (left) using a standard AR mask, Diffusion-VAR mask (middle) for hierarchical scales, BD3LM mask (right) adapted to hierarchical token counts.

In our hierarchical image setup, attention within the x_0 section is unnecessary and potentially counterproductive. Unlike BD3LM where clean tokens require contextualisation, the x_0 tokens already contain accumulated information from all previous scales through the residual structure $f_{k+1} = f_k - \phi_k(z_k)$. Each token in x_0 represents features that have been hierarchically refined and already incorporate the relevant context from coarser scales. Since $\phi_k(z_k)$ captures the portion of the feature map explained by quantised tokens at scale k , the remaining features in f_{k+1} inherently encode the unexplained residual information. Therefore, allowing x_0 tokens to attend to each other would be redundant as each token already possesses the necessary hierarchical context for predicting the target tokens from the noised input. Consequently, the block-causal component for $x_0 \rightarrow x_0$ is unnecessary, as the model can directly learn the mapping from x_0 to the target tokens without restricting attention among x_0 tokens themselves.

Experiments show that using a block-causal mask for $x_0 \rightarrow x_t$ improves training and validation accuracy, but reduces FID in comparison to a block-diagonal setup. As illustrated in Figure 3.1, the model input accumulates all relevant information from previous scales, which may explain the accuracy improvement: predictions benefit from preceding scales, but when teacher forcing is removed, errors accumulate and subsequent predictions are based on corrupted inputs.

The masking strategy consists of two complementary components:

1. **Teacher forcing from input:** Each block of noised tokens $x_t^{(i)}$ attends only to the ground-truth tokens $x_0^{(i)}$, ensuring that predictions at scale i are conditioned on correct tokens from preceding scales.
2. **Within-scale attention:** Each block of noised tokens has full self-attention within its own scale, enabling the model to capture dependencies among tokens at the same resolution.

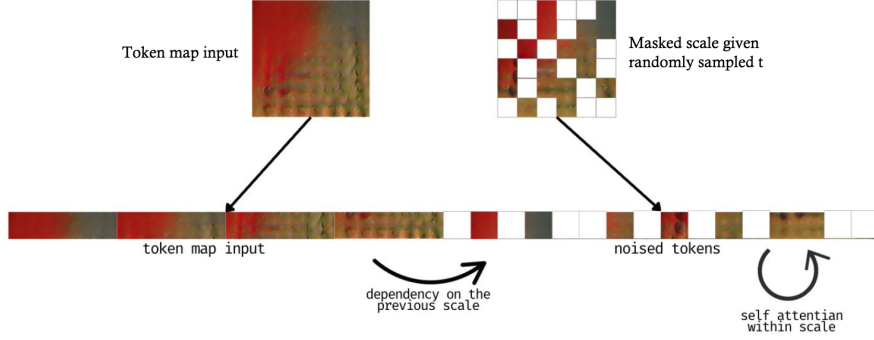


Figure 4.2: The relevant context for a specific scale, showing how a noised scale will depend on the clean input

The mask is converted into an attention bias by setting 0 to $-\infty$ and 1 to 0, which is then added to the attention scores. The softmax computation in multi-head attention zeroes prohibited connections, enforcing the desired hierarchical dependencies.

4.1.3 Learnable Mask Embedding

Each token in the VQ-VAE vocabulary is represented by a latent embedding of dimension $\mathcal{C}_{\text{VAE}}$. However, the VQ-VAE codebook does not include a representation for the mask token, which is required during the forward diffusion process. To address this, we introduce a dedicated learnable mask embedding of shape $(\mathcal{C}_{\text{VAE}})$, initialised with the same truncated normal scale `init_std` as the other embeddings in the VAR model to ensure magnitude consistency and stable training. During training, masked positions are replaced with this learnable embedding in $\mathcal{C}_{\text{VAE}}$, while all other tokens retain their standard VQ-VAE embeddings, so the sequence remains in the VQ-VAE latent space. The sequence is then projected into the diffusion model input space via the learned linear projection `word_embed`, allowing uniform processing of masked and unmasked tokens. At inference, sequences initially consist entirely of mask embeddings, which are iteratively refined by the diffusion process. Placing the mask embedding directly in the VQ-VAE latent space ensures compatibility with the pre-trained codebook and consistent representation magnitudes.

4.2 Noise Schedule and Masking Probabilities

4.2.1 Noise Schedule

The current implementation builds upon the MD4S codebase, which provides a flexible framework for masked diffusion with pre-defined noise schedules, including linear, polynomial, and cosine variants [23]. The cosine schedule smoothly interpolates between unmasked and fully masked states, gradually increasing masking probability toward the final timesteps. However, near $t = 0$, the noise schedule parameter β_t and the corresponding loss weight $w(t)$ can become extremely large, causing gradient instability and overemphasis on early timesteps during training.

4.2.2 Loss Weight Behaviour

For the continuous-time cosine masking schedule, the loss weight is defined as

$$w(t) = \frac{\alpha'(t)}{1 - \alpha(t)}, \quad \text{where} \quad \alpha(t) = 1 - \cos\left(\frac{\pi}{2}(1 - t)\right)$$

and $t \in [0, 1]$ with $t = 0$ representing fully clean tokens and $t = 1$ fully noisy tokens 2.31. so fully noisy tokens contribute very little to the gradient, as expected.

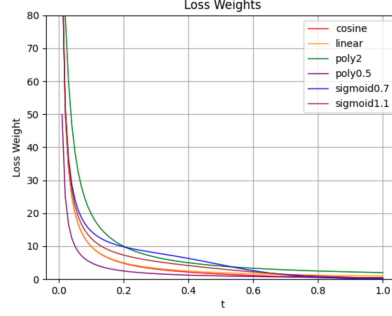


Figure 4.3: Loss weights for various noise schedules

During the **early stages** ($t \rightarrow 0$) the loss weight can be unstable as:

$$\alpha(0) = 1 - \cos\left(\frac{\pi}{2}\right) = 1, \quad \alpha'(0) = \frac{\pi}{2} \sin\left(\frac{\pi}{2}\right) = \frac{\pi}{2}, \implies w(0) = \frac{\pi/2}{0} \rightarrow \infty,$$

indicating that fully clean tokens dominate the gradient, which can cause instability if not clamped.

At **later stages** ($t \rightarrow 1$):

$$\alpha(1) = 1 - \cos(0) = 0, \quad \alpha'(1) = \frac{\pi}{2} \sin(0) = 0, \implies w(1) = 0,$$

so fully noisy tokens contribute very little to the gradient, as expected. Thus, the cosine

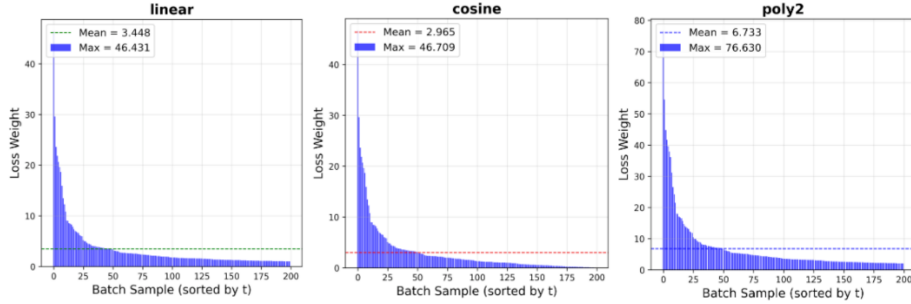


Figure 4.4: Distribution of loss weights across a randomly sampled batch of 200 timesteps, sorted. (Left) Linear schedule. (Middle) Cosine schedule. (Right) Polynomial with power 2).

schedule, among others, naturally assigns high gradient weight to early (mostly unmasked) tokens and low weight to late (fully masked) tokens. To mitigate this issue, we investigated two approaches: a `time_clamp` argument that prevents sampling too close to $t = 0$ and a `lw_max` argument that clips the maximum value of $w(t)$ to stabilize training. While time clamping initially appeared promising by avoiding problematic timesteps, it created a fundamental limitation where the model never learned to handle the final denoising steps effectively. Since these final steps require precise reconstruction with minimal room for error, excluding them from training resulted in poor inference performance when the model encountered these critical timesteps. After experimentation, we retained only the loss weight clipping approach. Loss weight clipping with the `lw_max` parameter proved more effective by ensuring that timesteps near $t = 0$ are still sampled and contribute to training while preventing extreme loss weights that cause gradient instability. For example, with

$\text{lw_max} = 35$, timesteps with $t > 0.1$ typically retain their original weights without clipping, while only the most extreme values near $t = 0$ are constrained. This approach preserves the intended emphasis on difficult reconstruction steps while maintaining numerical stability, resulting in more robust training dynamics. Concurrent work by [27] has investigated similar strategies, with systematic studies of different minimum time values finding optimal results at $t_{\min} = 0.2$, likely due to more stable gradient signals and better distribution of learning across the temporal range. Our loss weight clipping approach achieves similar benefits while maintaining full temporal coverage, ensuring that the model learns to handle all critical timesteps during inference.

4.2.3 Masking and Unmasking Probabilities

In the diffusion process, the noise schedule determines the probability of masking tokens during the forward process and the corresponding probability of reconstructing them during generation at each timestep t . Let $x_0 \in \mathbb{Z}^{B \times L}$ denote the ground-truth token sequence, augmented with an additional mask state m . The forward process gradually replaces tokens with the mask according to the schedule, while the reverse process reconstructs masked tokens conditioned on observed context. The exact derivation of these masking and unmasking probabilities is provided in 2.15.

4.3 Model Architecture and Conditioning

4.3.1 Class and Time Conditioning

The Visual Autoregressive (VAR) model employs only class conditioning, Diffusion VAR conditions on both time and class. Each class is associated with an embedding vector that is initialised using a truncated normal distribution with small variance, which ensures non-zero gradients during training. The class embedding is used in two ways. It provides the conditioning signal to the transformer, and it also serves as the start-of-sequence (SOS) token for the first level of hierarchical token generation. A learnable positional start parameter, `pos_start`, is added to the (SOS) token. This parameter acts as a bias that helps the model initialise suitable representations for autoregressive generation and introduces class-dependent priors for the sequence. Learnable timestep embeddings represent the temporal structure of generation. Timesteps are first mapped to sinusoidal frequency features, which are then passed through a multilayer perceptron to obtain embeddings. These embeddings provide the model with information about the progression of generation, ensuring consistency across denoising and autoregressive steps. Class embeddings establish the semantic context of the output, while timestep embeddings structure the generative process over time.

4.3.2 Positional Encoding

As implemented the original VAR design, the model incorporates several forms of positional encoding to provide the model with information about spatial arrangement, hierarchical level, and the clean starting position of tokens. Absolute position embeddings encode spatial structure within each resolution level. For every level in the hierarchy, a separate embedding tensor is initialised with truncated normal noise and then concatenated across levels, resulting in a global positional embedding of shape $(1, L, C)$. This embedding provides the model with explicit positional information for each token, ensuring that spatial relationships are preserved throughout the autoregressive process. The model includes a level embedding that functions similarly to the segment embedding used in GPT models [35, 4]. Each scale is assigned a learnable vector through an embedding, allowing the model

to distinguish between tokens originating from different scales. Both absolute and level embeddings are learnable, allowing the model to adaptively capture spatial and hierarchical structure during training. Consequently, I did not incorporate the commonly used two-dimensional sinusoidal positional encodings that appear in vision transformers, such as in the MD4 and DiT models [23, 48].

4.3.3 Backbone Architecture.

The backbone of our model is a transformer architecture derived from the Visual Autoregressive (VAR) framework, adapted for conditional diffusion modelling. However, is DiT-like in that it incorporates transformer blocks with self-attention conditioned on time and class embeddings. Conditioning is applied through adaptive layer normalisation (AdaLN), which computes shift, scale, and gating parameters from the conditioning vectors and modulates both the attention and feed-forward layers. This allows the model to dynamically adapt its feature processing based on timestep and class information. Although we implemented and tested a direct Diffusion Transformer (DiT) [48] backbone, it was outperformed by our modified transformer derived from the Visual Autoregressive (VAR) framework.

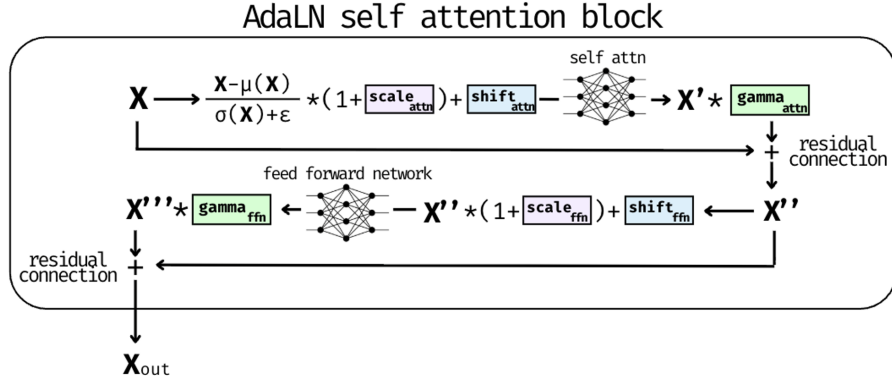


Figure 4.5: The process within the AdaLN self attention block.

Each transformer block follows the `AdaLNSelfAttn` design, consisting of multi-head self-attention and feed-forward networks, both modulated by conditioning inputs through adaptive layer normalisation. Optional optimisations, including FlashAttention [49] and XFormers memory-efficient attention, are used when available to improve computational efficiency. The self-attention mechanism allows the model to capture dependencies across tokens in the sequence, both within and between scales, optionally guided by an attention mask. Optional normalisation and learnable scaling are applied to improve stability. Temporal conditioning is handled by a dedicated `TimestepEmbedder` module and follows a standard implementation. Scalar timesteps are transformed into sinusoidal frequency embeddings [33], which are then processed through a small multilayer perceptron to produce dense representations. Class and timestep embeddings are combined through element-wise addition to form a unified conditioning vector. This vector modulates both the self-attention and feed-forward layers via adaptive layer normalisation. Each block computes shift, scale, and gating parameters from the conditioning vector, which dynamically adjust the normalisation and residual connections. The model is initialised with the initial standard deviation defined in VAR, and modulation layers initially zeroed to facilitate stable training. The backbone operates on sequences of fixed length and does not directly manage positional or class embeddings, which are instead handled by the diffusion model. This separation allows the diffusion model to track variable patch sizes across training and inference, providing precise control over hierarchical token representations while keeping

the transformer’s input and output sequence dimensions consistent.

4.3.4 Adaptive Layer Normalisation Implementation

The class conditioning is integrated throughout the transformer backbone via Adaptive Layer Normalisation (AdaLN) layers [50, 48]. A critical observation from VARs work is that normalising queries and keys to unit vectors before attention computation can significantly stabilise training dynamics [13]. AdaLN has demonstrated effectiveness across various generative modelling domains, including language modelling applications in conditional text generation [36] and visual synthesis tasks [15, 51].

Following the empirical findings in DiT [48], AdaLN has become widely adopted in masked diffusion models [23, 22, 27]. Two primary approaches exist for combining conditioning signals within AdaLN frameworks. The original DiT processes class and time embeddings through separate linear projections before combining them additively to generate conditioning vectors. In contrast, MD4 concatenates class and time embeddings into a joint representation, which is then processed through a unified MLP to produce AdaLN modulation parameters.

Both approaches demonstrate strong empirical performance, with distinct trade-offs. Additive combination requires fewer parameters and assumes conditioning signals operate in compatible representational spaces. Concatenation provides flexibility by allowing the MLP to learn interactions between conditioning modalities, though at increased computational and parameter cost. Following the DiT approach [48], I chose to combine class and timestep embeddings via simple addition, while positional embeddings in VAR remain additive and class embeddings are handled via AdaLN.

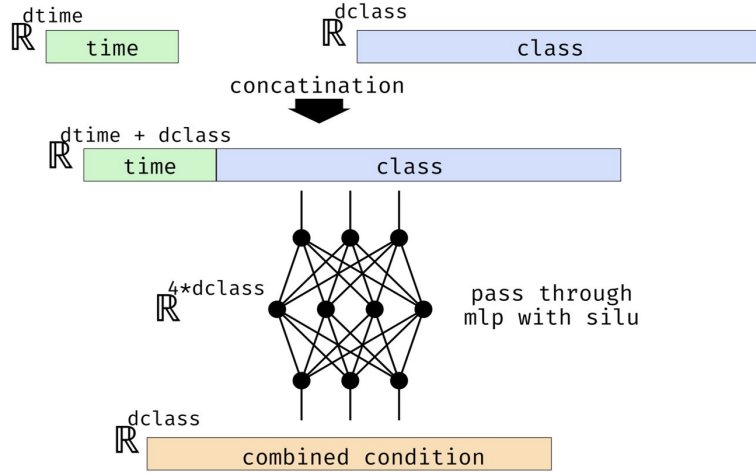


Figure 4.6: The time and class embedding are combined with a MLP to produce a combined condition in d_{embed} , the models conditioning dimension.

Multi-Scale Resolution Challenges In hierarchical diffusion–autoregressive models, generation is costly because every resolution must be produced sequentially. At each scale, the model performs a full diffusion process conditioned on all lower-resolution outputs. This compounds computation compared to standard diffusion, where only the final resolution is denoised. Autoregressive generation further increases complexity, as each token depends on the preceding context. Naïvely, this would require reprocessing the entire sequence at every step. However, in our setup the enforced masking allows the model to condition only on the single accumulated token map from previous scales. This significantly reduces the

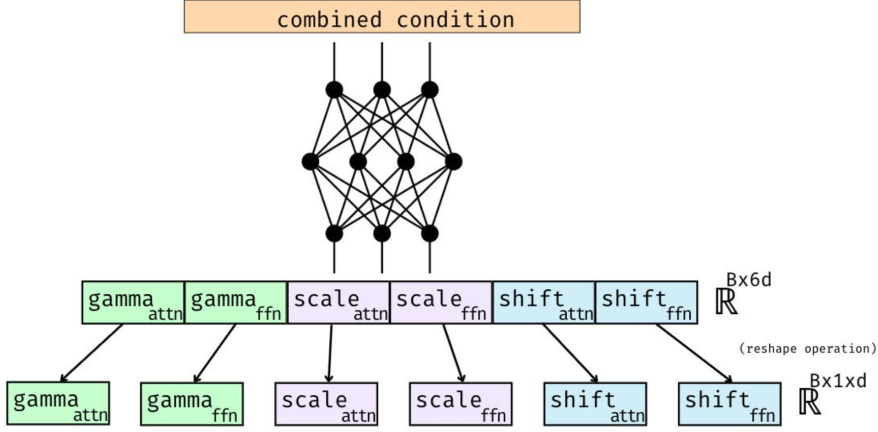


Figure 4.7: The AdaLN parameters are predicted conditioned on the embedding.

effective input length from a sequence of size $2 \sum_{n=1}^K p_n^2$ to $2p_n^2$, where p_n is the number of patches per side, rather than the full history of all generated tokens.

While key-value (KV) caching would reduce computation by reusing attention projections [52] (by caching the input token map), AdaLN conditioning depends on the timestep t , which changes at every denoising step. This makes cached key-value pairs time-dependent and invalid once t is updated. Hypothetically, if we could use key-value caching it would result in $\mathcal{O}(n^4T)$ scaling. However due to constraints we do not and the computation scales as $\mathcal{O}(n^4T)$. Even if we processing the full sequence, the asymptotic behaviour would not change, as derived in 6.18, but restricting the input to $2p_n^2$ tokens is significantly more efficient in practice.

Alternative Conditioning Approaches and Training Instability Both cross-attention and in-context conditioning were explored as alternatives to AdaLN, in the hope of producing a formulation where the conditioning input was time-independent, enabling KV caching. Cross-attention allocates separate heads for timestep and class information, while in-context conditioning appends conditioning tokens directly to the sequence. In our experiments, an attention mask was applied to restrict where the timestep conditioning could influence the sequence, but both methods proved unstable: gradients steadily diminished, leading to vanishing updates and ineffective conditioning. Likely contributing factors include mismatched normalisation statistics between conditioning and image tokens, scale disparities in their embeddings, and poor gradient flow when early layers fail to attend to conditioning tokens. As these approaches would have required extensive tuning and diagnostics to stabilise, they were not further pursued. Instead, only AdaLN was adopted.

4.4 Sampling

The generation process operates through a sequence of resolution scales, where each scale corresponds to a different patch granularity of the target image. The model processes these scales sequentially, with each subsequent scale dependent on the representations generated in all preceding scales. This hierarchical dependency ensures that each scale generates a refinement of the image rather than reconstructing the entire map, allowing finer scales to focus on details not captured by coarser scales. By progressively adding these refinements, the model maintains global coherence while incrementally improving local detail.

Adaptive Number of Time-steps A design choice in hierarchical sampling is the use of adaptive timestep scheduling that varies according to the number of tokens in each patch scale. Without such scaling, early scales are unnecessarily expensive: for example, in the first scale containing a single token, a cosine noise schedule requires $t \approx 0.35$ to reach a cumulative noise probability of 0.5, meaning that unmasking a single token is likely to undergo more than half of the total sampling steps that we want to use on the final scale.

To address this, two methods for adaptive scheduling. are proposed.

- **Linear:** steps proportional to patch count:

$$\text{steps}_{\text{linear}} = \max \left(1, \lfloor \text{patch_length} \cdot \frac{\text{num_steps}}{\text{patch_length}_{\text{max}}} \rfloor \right).$$

- **Logarithmic:** more conservative scaling for small patches:

$$\text{steps}_{\text{log}} = \max \left(1, \lfloor \log(\text{patch_length} + 1) \cdot \frac{\text{num_steps}}{\log(\text{patch_length}_{\text{max}} + 1)} \rfloor \right).$$

Linear scaling is simple but can under-allocate steps for small patches, reducing denoising quality. Logarithmic scaling is more conservative, ensuring small scales receive enough steps while maintaining efficiency. While lower scales contain fewer tokens, each token carries proportionally more information, therefore, waiting to unmask these tokens until the model attains a confident prediction improves denoising quality. For example, a scale with 16 tokens receives 38 steps under logarithmic scaling compared to 25 under linear scaling, while the largest scale with 64 tokens retains the full 100 steps.

Token Sampling and Embedding Updates

At each diffusion step t , the model outputs logits $f_{\theta}(x_t, t)$ over the extended vocabulary, which are processed through classifier-free guidance before sampling. Classifier-free guidance (CFG) [51] is applied to improve semantic alignment with class conditioning. The CFG-adjusted logits are computed as

$$\text{logits} = (1 + y) \cdot \text{logits}_{\text{cond}} - y \cdot \text{logits}_{\text{uncond}}, \quad y = \text{cfg} \cdot \frac{\text{scale_index}}{\text{num_scales} - 1}.$$

y provides scale-adaptive guidance strength. This adaptive guidance increases in later scales, where fine details benefit more from conditioning, while early scales retain flexibility to establish global structure.

The predictive distribution over vocabulary tokens is obtained by applying softmax to the CFG-adjusted logits, excluding the [MASK] dimension:

$$\mu_{\text{vocab}}(x_t, t) = \text{softmax}(f_{\theta}(x_t, t)_{1:V}).$$

Following the D3PM [29] framework, the sampling distribution combines vocabulary predictions with schedule-defined masking probabilities. The unmask probability is computed as

$$p_{\text{unmask}} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t},$$

where α_t and α_s are consecutive schedule values, $s < t$, as t goes from $t = 1.0$ to $t = 1/\text{num_steps}$. We do not need to process $t=0$ as at that point the image should be clean. At the final step $s = 0$ therefore $\alpha_s = 1$ and the $p_{\text{unmask}} = 1$. In practice, due to floating point error, the probability for the last stage is manually set to 1, to avoid mask tokens remain in the sample. Top- k and nucleus (top- p) filtering or temperature scaling may be applied to $\mu_{\text{vocab}}(x_t, t)$ before re-weighting according to the unmask probability, controlling

the trade-off between generation diversity and fidelity. The final sampling distribution is constructed as:

$$\tilde{\mu}_{\theta}(x_t, t) = p_{\text{unmask}} \cdot \mu_{\text{vocab}}(x_t, t) \oplus (1 - p_{\text{unmask}}) \cdot \mathbf{1}_{[\text{MASK}]},$$

where $\mathbf{1}_{[\text{MASK}]}$ denotes the one-hot vector for the mask token.

Categorical sampling from $\tilde{\mu}_{\theta}(x_t, t)$ produces candidate tokens, which are then filtered through a carry-over constraint that preserves previously unmasked positions. Only currently masked positions $\{i : x_t^{(i)} = [\text{MASK}]\}$ can be updated with new token predictions. For newly unmasked positions where $\text{sampled}_i \neq [\text{MASK}]$, the corresponding input embeddings are immediately updated to the VQ-VAE quantised representations, ensuring that subsequent diffusion steps operate on the updates, and remaining masked positions continue to propagate as placeholders until resolved in later steps.

Inter-Scale Information Propagation

After completing the diffusion process for scale s_i , the generated tokens are mapped through the VQ-VAE embedding lookup and spatially reshaped to match the patch grid dimensions:

$$h_{s_i} = e(\text{tokens}_{s_i}), \quad h_{s_i} \in \mathbb{R}^{B \times C_{\text{VAE}} \times p_i \times p_i},$$

where $e(\cdot)$ denotes the VQ-VAE codebook embedding and p_i is the patch resolution at scale s_i . These embeddings are then processed to accumulate features and prepare conditional inputs for the subsequent scale. The residual accumulation operation takes as input the current scale index, the accumulated features, and the current scale embeddings

$$s_i, \quad f_{s_i} \in \mathbb{R}^{B \times C_{\text{VAE}} \times H_{\text{max}} \times W_{\text{max}}}, \quad h_{s_i} \in \mathbb{R}^{B \times C_{\text{VAE}} \times p_i \times p_i}$$

For non-final scales, the current embeddings are upsampled to the maximum resolution via bicubic interpolation, processed through a scale-specific residual convolutional layer, and added to the accumulated features. The function returns the updated accumulated features and the conditional context for the next scale, which obtained by downsampling the accumulated features to the resolution of scale s_{i+1} using area interpolation.

$$f_{s_{i+1}} \in \mathbb{R}^{B \times C_{\text{VAE}} \times H_{\text{max}} \times W_{\text{max}}}, \quad \text{next_token_map}_{s_{i+1}} \in \mathbb{R}^{B \times C_{\text{VAE}} \times p_{i+1} \times p_{i+1}},$$

These representations are then transformed and combined with positional and level embeddings to create the conditioning input:

$$\text{input}_{s_{i+1}} = \text{WordEmbed}(\text{input}_{s_{i+1}}) + \text{LevelEmbed}(s_{i+1}) + \text{PosEmbed}(s_{i+1})$$

where level embeddings encode scale-specific information and positional embeddings capture spatial relationships within the patch grid, and clean positional embeddings indicate a complete scale. For the final scale, embeddings are processed without upsampling, and the accumulated features serve as both the final output and the conditional context. This hierarchical accumulation ensures that each scale has access to all previously generated information through the conditional context x_0 , while maintaining the autoregressive dependency chain across resolution levels.

4.5 Implementation and Experiments.

Initially, I began with the BD3LM codebase [47], which implements a blocked diffusion mask for language modelling. This framework allowed me to map word-part tokens in

language sequences to the hierarchical token sequences in VAR [13], treating coarser image scales as analogous to larger text blocks. To adapt this setup for image diffusion, I integrated timestep embeddings following RoPE [?] and incorporated conditioning through zero-initialised AdaLN layers, modifying the VAR transformer to accept these augmented inputs. Early experiments with this approach showed moderate improvements in stability and gradient flow, but the resulting generative quality and FID scores remained lower than VAR’s original performance. This highlighted key differences between transformers optimised for diffusion versus autoregressive modelling.

To investigate diffusion-specific backbones, I implemented the Diffusion Transformer (DiT) exactly as described in the original paper [48], including adaptive layer normalisation, timestep and class conditioning, and multi-head self-attention blocks. Despite faithfully reproducing DiT, the results were consistently inferior to the modified VAR transformer. In particular, the DiT backbone struggled with gradient stability on hierarchical sequences and produced higher FID scores compared with our VAR-inspired design. I also experimented with MD4 [23]’s transformer backbone, which offers bidirectional attention, flexible sequence handling, and explicit timestep conditioning. MD4 performed better than DiT in terms of training stability, but the modified VAR transformer, with its shared AdaLN conditioning, zero-initialisation strategies, and autoregressive structure, ultimately yielded the best combination of stability, convergence speed, and generative fidelity.

These experiments demonstrate that while DiT and MD4 provide valuable insights for diffusion-specific architectures, careful adaptation of the VAR transformer to incorporate timestep and class conditioning through zero-initialised AdaLN layers offered an effective backbone for hierarchical image generation.

Chapter 5

Results

5.1 Experimentaion Set Up

Both VAR and DiffusionVAR models were trained under comparable conditions on the ImageNet-128 subset with 10 classes ($\sim 13\text{k}$ images), using the pretrained VQ-VAE from VAR [13] with a 4096-codebook and 32-channel latent space, where each token corresponds to a 16×16 image patch. Training was performed for 250 epochs on 2 NVIDIA A16 GPUs with a global batch size of 128, taking roughly 7 hours. For optimisation, AdamW we used with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay = 0.05, and gradient clipping at 2.0. Learning rates were scaled linearly with batch size relative to the reference configuration: DiffusionVAR used a peak learning rate of 0.0001 (base rate $0.0002 \times 128/256$) for base batch size 256, Both schedules followed the same warmup–decay scheme: linear warmup from 0.005 to 1.0 over 5 epochs, a flat phase for the first 5% of training, then linear decay to 0.01 across the remaining 95%.

DiffusionVAR was trained under the continuous-time formulation using with a cosine noise schedule. For stability the loss weights were clipped to 6. Inference was performed with 50 diffusion steps. Both models were trained with hierarchical patch configurations [1, 2, 3, 4, 5, 6, 8].

Evaluation was conducted every few epochs using cross-entropy loss and classification accuracy on the validation set. Generative quality was monitored with FID, LPIPS, and Inception Score, computed on 1k sampled images. While 1k is relatively small and less accurate than the standard 10k+ evaluation, it was sufficient to track performance trends during training, as larger scale evaluations would have been excessively costly to compute.

5.2 Evaluation Framework and Metrics

The evaluation framework combines token-level metrics (cross-entropy and accuracy) with perceptual image metrics (FID, IS, LPIPS) to assess both reconstruction fidelity and generative quality. For DiffusionVAR, the training loss uses a weighted denoising objective across diffusion steps, while the logged evaluation loss is standard cross-entropy over masked tokens.

Image-level evaluation relies on FID, IS, and LPIPS, each capturing different aspects of image quality. FID (Fréchet Inception Distance) measures the similarity between the distributions of real and generated images in a high-level feature space extracted from a pretrained InceptionV3 network. Specifically, it computes the mean and covariance of these features and calculates the Fréchet distance between them. Lower FID indicates that generated images closely match the real data distribution, reflecting both realism and diversity. IS (Inception Score) evaluates the quality and variety of generated images using the conditional and marginal class probabilities predicted by an Inception classifier. It

favors images that are individually sharp and easily classifiable (low-entropy conditional labels) while ensuring that the generated set covers many semantic classes (high-entropy marginal distribution). Both FID and IS rely on InceptionV3, pretrained on ImageNet, which provides a robust and widely accepted feature space for comparing image content and diversity. LPIPS (Learned Perceptual Image Patch Similarity) measures perceptual similarity between image pairs by comparing deep feature activations from a pretrained network. Unlike simple pixel-wise distances, LPIPS correlates strongly with human perception, capturing differences in texture, structure, and fine details. This is especially important in conditional generation, where the alignment between reconstructed images and ground-truth targets is critical.

5.3 Quantitative Metrics

Table 5.1: Evaluation metrics

Model	FID ↓	LPIPS ↓	Inception Score ↑	time (s)
DiffusionVAR (ours)	178.0	0.6572	1.074	53.1
VAR (baseline)	96.8	0.6536	1.0803	0.9

Table 5.2: Time to generate 1 batch (128 images, 1 GPU, 50 time steps, 8 scales)

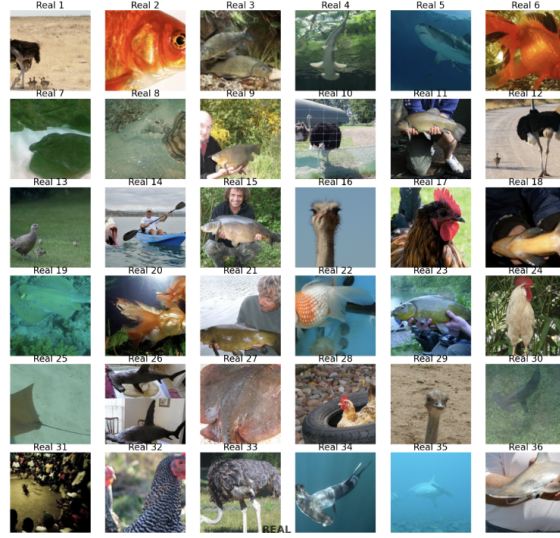
Model	method	time (s)	total steps
DiffusionVAR	no reduction	53.01	400
DiffusionVAR	linear	34.01	120
DiffusionVAR	logarithmic	42.78	219
VAR (baseline)	AR	0.6536	8

We observe that VAR outperforms DiffusionVAR in both generative quality and computational efficiency. Across evaluation metrics, VAR achieves higher reconstruction fidelity at the token level and produces images with lower FID and higher IS scores, indicating more realistic and semantically diverse outputs. In addition to these quality gains, VAR demonstrates faster inference and lower resource usage, likely due to its more streamlined autoregressive architecture compared to the multi-step denoising process in DiffusionVAR. DiffusionVAR is slower because each patch scale is not decoded once, but iteratively refined over many diffusion timesteps. At every step it recomputes noise and time embeddings, evaluates unmasking probabilities, runs the backbone under classifier-free guidance, and resamples tokens, so the same overheads that VAR pays once per scale are repeated across the entire diffusion schedule. Importantly, this runtime does not scale linearly with the number of steps: early steps operate on short sequences while later steps operate on much longer ones, so reducing the step count disproportionately removes cheap iterations and leaves expensive ones largely intact. As a result, even with step reductions, DiffusionVAR remains consistently slower than purely autoregressive decoding. These results suggest that while diffusion-based approaches provide flexibility in modeling complex distributions, the autoregressive VAR framework remains highly effective for tasks where efficiency and high-quality reconstructions are critical, highlighting the need to balance model expressiveness with practical inference considerations.

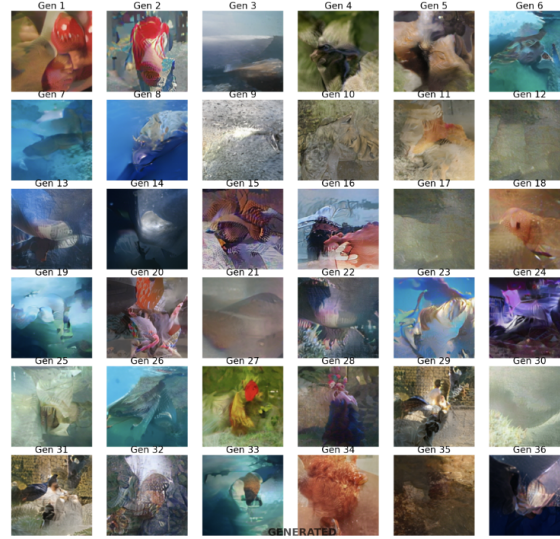
DiffusionVAR runs slower than VAR because each patch scale is not decoded once, but iteratively refined over many diffusion timesteps. At every step it recomputes the

noise and time embeddings, evaluates unmasking probabilities, runs the backbone under classifier-free guidance, and resamples tokens, so the same overheads that VAR pays once per scale are repeated across the entire diffusion schedule. Importantly, this runtime does not scale linearly with the number of steps: early steps operate on short sequences while later steps operate on much longer ones, so reducing the step count disproportionately removes cheap iterations and leaves expensive ones intact. As a result, lowering the nominal step budget yields sublinear speedups, and DiffusionVAR remains consistently slower than purely autoregressive decoding.

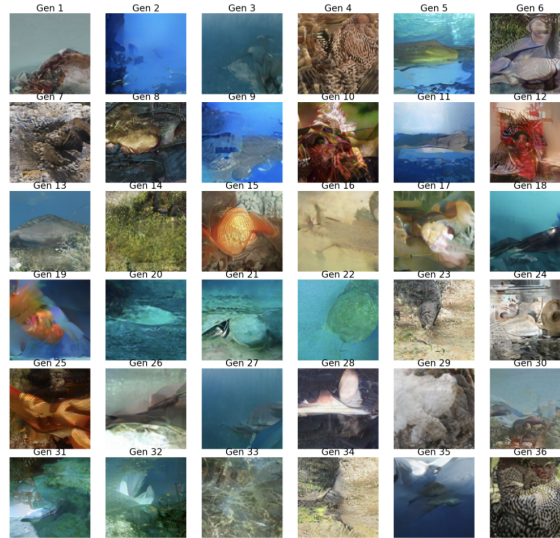
5.3.1 Image Comparison



(a) Real Image



(b) DiffusionVAR FID:178.0



(c) VAR FID:96.8

Figure 5.1: Qualitative comparison of generated images.

5.3.2 Class Conditional Generation

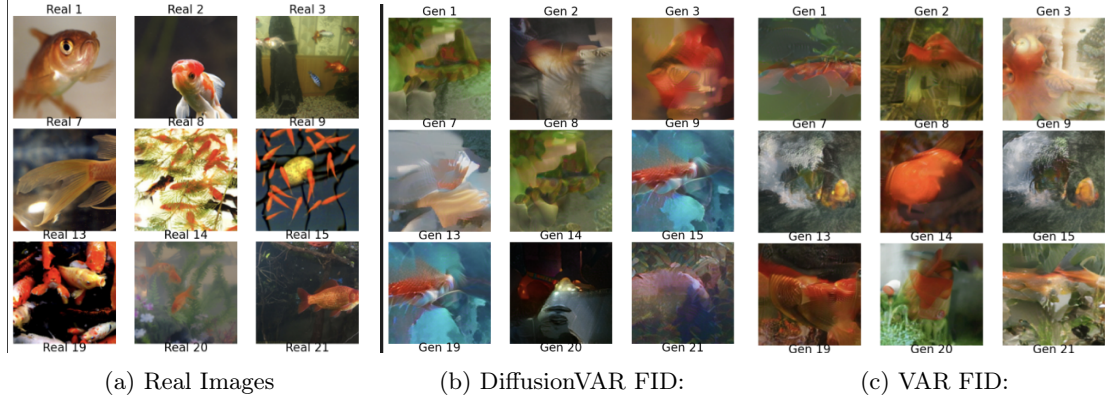


Figure 5.2: Generated Images Conditioned on Class 1

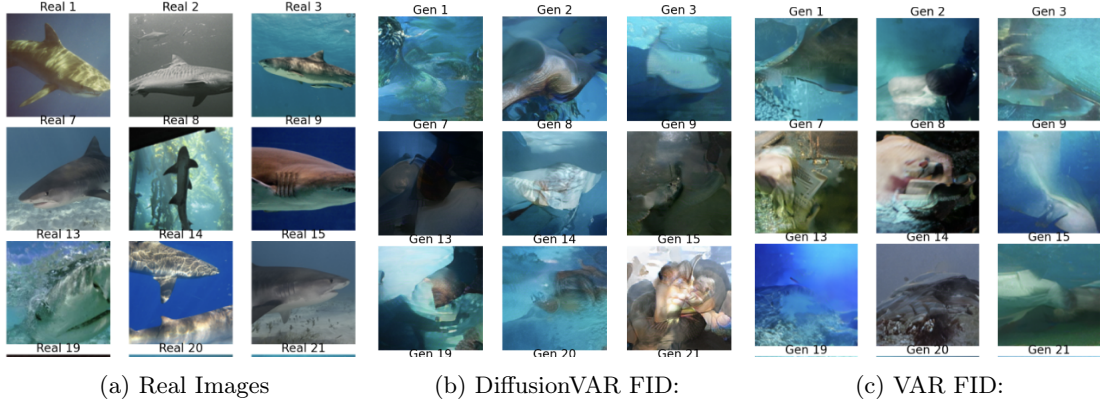


Figure 5.3: Generated Images Conditioned on Class 3

5.4 Qualitative Evaluation

In the experiments, VAR consistently achieves higher accuracy during both training and evaluation compared to DiffusionVAR. This improved performance is observed at the token level, reflecting more accurate reconstruction and prediction. Notably, VAR is trained using a standard cross-entropy loss, which directly optimises the likelihood of the correct tokens, providing a training signal that contributes to its superior accuracy.



Figure 5.4: Evaluation Accuracy through training for the last 6 (/7) scales. VAR (blue), DiffusionVAR (red)

Chapter 6

Complexity Analysis

Notation. For simplicity, we assume all images have shape $H \times W \times C$, and further set $H = W = n$, so that the number of spatial positions is $m = n^2$. Let d denote the model’s hidden dimension and h the number of attention heads. In transformer-style architectures, the self-attention operation computes pairwise interactions between all tokens in the input sequence of length m , producing attention weights and weighted sums for each head. The full computational complexity of self-attention is therefore

$$\mathcal{O}\left(h m (m d_h + d_h^2)\right) = \mathcal{O}(m^2 d + m d^2),$$

where $d_h = d/h$ is the head dimension. The $m^2 d$ term arises from computing all pairwise attention scores and applying them to the value vectors, while the $m d^2$ term comes from the subsequent linear projections. In practice, for large-scale image generation we are primarily concerned with the scaling of the attention operation with respect to sequence length m , and we will assume that d is fixed. Consequently, in the following analysis we simplify the self-attention complexity to $\mathcal{O}(m^2)$ and focus on how m varies across different architectures and representations, providing a consistent framework for comparing autoregressive, latent, and diffusion-based image generation models.

6.1 Autoregressive Image Models

Raster-Scan Autoregression. In the standard autoregressive formulation, an $n \times n$ RGB image is unfolded into a sequence of $m = 3n^2$ tokens, each drawn from a vocabulary of size $V = 256$ corresponding to 8-bit pixel intensities. Training proceeds with teacher forcing at cost $\mathcal{O}(m^2) = \mathcal{O}(n^4)$, since all tokens can be processed in parallel.

Proof of training complexity: Under teacher forcing, the model sees the full sequence of length m . Summing over all token positions gives

$$S = \sum_{k=1}^m k = \frac{m(m+1)}{2}. \quad (6.1)$$

Squaring this to account for attention operations yields the total training complexity:

$$\mathcal{C}_{\text{train}} = S^2 = \left(\frac{m(m+1)}{2}\right)^2 = \frac{m^2(m+1)^2}{4} = \mathcal{O}(m^4) = \mathcal{O}(n^4), \quad (6.2)$$

where $m = 3n^2$ for an $n \times n$ RGB image.

Proof of generation complexity: Sequential generation requires predicting one token at a time. At step k , the cost is k^2 , so total generation cost is

$$\mathcal{C}_{\text{gen}} = \sum_{k=1}^m k^2 = \frac{m(m+1)(2m+1)}{6} = \mathcal{O}(m^3) = \mathcal{O}(n^6). \quad (6.3)$$

With KV caching, each step only computes attention against previously computed key–value pairs, reducing the per-step cost to k , giving

$$\mathcal{C}_{\text{KV}} = \sum_{k=1}^m k = \frac{m(m+1)}{2} = \mathcal{O}(m^2) = \mathcal{O}(n^4). \quad (6.4)$$

For a 256×256 image, this corresponds to $m = 196,608$ tokens, illustrating why pixel-level autoregression is computationally expensive despite its conceptual simplicity.

Latent Space Autoregression. To reduce sequence length, VQ-VAE compression maps images to a discrete latent grid of size $(n/r)^2$, where r is the downsampling factor. The original VQVAE encodes a $128 \times 128 \times 3$ image into a $32 \times 32 \times 1$ discrete latent space with code size 512 [3].

Training and generation with KV caching scale as $\mathcal{O}((n/r)^2)$, while ordinary sequential generation without caching has cost $\mathcal{O}((n/r)^3)$. This reflects a reduction by a factor of up to r^6 relative to pixel-space autoregression. Latent autoregression achieves savings of several orders of magnitude in practice and is widely used for large-scale generative models.

Visual AutoRegressive (VAR) Model. VAR [13] replaces flat sequences with a hierarchical multi-resolution generation process. An image of size $n \times n$ is factorised into K stages with resolutions $(n_1, n_1), (n_2, n_2), \dots, (n_K, n_K)$. Let $a > 1$ be the upsampling factor such that $n_k = a^{k-1}$ and $n = a^{K-1}$. At each stage, the model predicts n_k^2 tokens. Training retains cost $\mathcal{O}(n^4)$ under teacher forcing.

Proof of training complexity: The cumulative number of tokens across all stages is

$$M_{\text{train}} = \sum_{k=1}^K n_k^2 = \sum_{k=1}^K a^{2(k-1)} = \frac{a^{2K} - 1}{a^2 - 1}. \quad (6.5)$$

Self-attention scales quadratically in the number of tokens, giving training cost

$$\mathcal{C}_{\text{train}} = \mathcal{O}(M_{\text{train}}^2) = \mathcal{O}\left(\left(\frac{a^{2K} - 1}{a^2 - 1}\right)^2\right). \quad (6.6)$$

Substituting $a^K = a \cdot n$ yields

$$\mathcal{C}_{\text{train}} = \mathcal{O}\left(\frac{(an)^4 - 1}{(a^2 - 1)^2}\right) = \mathcal{O}(n^4), \quad (6.7)$$

so the forward pass scales as $\mathcal{O}(n^4)$, since a is constant.

Proof of generation complexity: At stage k , each token must attend to all previously generated tokens from stages 1 through k . The cumulative number of tokens up to stage k is

$$M_k = \sum_{i=1}^k n_i^2 = \sum_{i=1}^k a^{2(i-1)} = \frac{a^{2k} - 1}{a^2 - 1}, \quad \text{Cost}_k = M_k^2 = \left(\frac{a^{2k} - 1}{a^2 - 1}\right)^2. \quad (6.8)$$

Summing over all stages gives the total generation cost:

$$\mathcal{C}_{\text{gen}} = \sum_{k=1}^K \left(\frac{a^{2k} - 1}{a^2 - 1}\right)^2. \quad (6.9)$$

Expanding the sum:

$$\mathcal{C}_{\text{gen}} = \sum_{k=1}^K \frac{a^{4k} - 2a^{2k} + 1}{(a^2 - 1)^2} = \frac{1}{(a^2 - 1)^2} \sum_{k=1}^K (a^{4k} - 2a^{2k} + 1). \quad (6.10)$$

Using geometric series and $K \sim \log_a n$, the largest term a^{4K} dominates, giving

$$\mathcal{C}_{\text{gen}} \sim \frac{a^{4K} - 1}{(a^2 - 1)^2(a^4 - 1)} \sim O(n^4), \quad (6.11)$$

which is substantially more efficient than raster-scan autoregression’s $O(n^6 d)$ [13].

Generation with KV caching: Reusing previously computed key-value pairs reduces per-stage attention cost to

$$\mathcal{C}_{\text{KV}} = n_k^2 \cdot \sum_{i=1}^{k-1} n_i^2 = a^{2(k-1)} \cdot \frac{a^{2(k-1)} - 1}{a^2 - 1} = \frac{a^{4(k-1)} - a^{2(k-1)}}{a^2 - 1}. \quad (6.12)$$

Summing over all stages gives total KV-cached generation cost:

$$\mathcal{C}_{\text{KV}} = \sum_{k=1}^K \frac{a^{4(k-1)} - a^{2(k-1)}}{a^2 - 1}. \quad (6.13)$$

The geometric series dominated by $a^{4K} \sim n^4$, giving $\mathcal{C}_{\text{KV}} \sim O(n^4)$. While KV caching provides significant constant factor improvements, it does not change the asymptotic scaling due to the hierarchical generation structure where each stage still requires attention computation proportional to the cumulative token count.

6.2 Diffusion Models

Diffusion models operate in continuous space $\mathbb{R}^{h \times w \times c}$, learning to denoise progressively corrupted images. Each denoising step applies a backbone network, such as a transformer or a UNet-like convolutional model.

Training complexity is determined by the single-step denoising objective and is parallelised over all pixels, resulting in $\mathcal{O}(n^4 c^2 d)$ for attention-based models or $\mathcal{O}(n^2 c P)$ for convolutional models. The main computation is the quadratic attention over spatial positions in transformers or the linear but repeated convolutions in UNets. Training complexity is independent of the total number of denoising steps T , since each step is optimised separately.

Unlike autoregressive models, KV caching cannot be applied because each pixel changes at every denoising step and predictions are explicitly conditioned on the timestep.

At *generation* time, each denoising step applies the backbone to all pixels. The per-step cost is the same as during training, but it must be repeated T times, giving total generation complexity

$$\mathcal{O}(T \cdot n^4 c^2 d)$$

for transformers and

$$\mathcal{O}(T \cdot n^2 c P)$$

for convolutional backbones. Typical diffusion models use $T \ll 10^3$ steps. Diffusion models are generally faster than pixel-level autoregressive models at high resolutions due to parallelised updates and smaller per-step sequences, but slower than latent-space autoregressive models when T is large or when attention is quadratic in n^2 . KV caching cannot be applied in diffusion generation, unlike in autoregressive models.

For a 256×256 RGB image, the attention-based transformer backbone would refine $n^2 c = 196,608$ tokens per step, with T steps amplifying the cost linearly. Diffusion models are typically faster than pixel-level autoregressive models at high resolutions due to the parallelisable training and smaller per-step sequence length, but slower than latent-space autoregressive models when T is large or attention is quadratic in n^2 or when using KV

caching. Comparing for the same $n \times n \times c$ image, diffusion scales as $\mathcal{O}((nc)^4 dT)$ while pixel-level autoregression scales as $\mathcal{O}((nc)^6 d)$; for example, with a $256 \times 256 \times 3$ image, diffusion generation is faster whenever $T < 3.8 \times 10^{10}$ steps (typical values are $T \ll 10^3$), though when KV caching is applied, autoregression becomes comparatively more efficient. TODO

Diffusion Transformers (DiTs) DiTs extend diffusion to patch-based token sequences, reducing sequence length from $h \cdot w$ pixels to $m = (h \cdot w)/p^2$ patches. Training thus scales as $\mathcal{O}(m^2 d)$, and generation requires $\mathcal{O}(T \cdot m_p^2 d) = \mathcal{O}(T \cdot (h \cdot w)^2 / p^4 d)$. For a 256×256 image with patch size $p = 16$, the sequence length reduces from 65,536 pixels per channel to $m = 256$ patches, giving generation cost $\mathcal{O}(T \cdot 256^2 d)$ rather than $\mathcal{O}(T \cdot 65,536^2 d)$, a quadratic reduction in effective sequence length. The dominant factor remains the T denoising iterations, though patching provides a decisive efficiency gain. TODO

Masked Diffusion Models (MDMs) MDMs operate similarly to standard diffusion models but in discrete latent spaces resembling latent autoregressive models but with iterative refinement over a sequence of length m and vocabulary size K . Training employs masked denoising objectives and scales as $\mathcal{O}(m^2 d)$, since full self-attention is applied across all tokens. Generation requires T refinement steps, each involving $\mathcal{O}(m^2 d)$ complexity, yielding an overall cost of $\mathcal{O}(T \cdot m^2 d)$. The quadratic self-attention at each step constitutes the dominant bottleneck, making MDMs significantly more expensive than autoregression in practice despite their conceptual similarity to latent models.

6.3 Proposed Model

Diffusion VAR. The multiscale diffusion autoregressive model generates images of resolution $n \times n$ hierarchically through K stages, where stage k has resolution $n_k = a^{k-1}$ and n_k^2 tokens, with upsampling factor $a > 1$ and $n = a^{K-1}$. To balance computational effort across scales, the number of diffusion steps per stage is scaled according to the number of tokens at that stage. While both logarithmic and linear scaling strategies are implemented, we demonstrate using a fixed T which serves as an upper bound.

For training, the cumulative number of tokens is

$$M_{\text{train}} = \sum_{k=1}^K 2n_k^2 = 2 \sum_{k=1}^K a^{2(k-1)} = 2 \frac{a^{2K} - 1}{a^2 - 1}, \quad (6.14)$$

and since we use a single sampled timestep per iteration, this does not depend on T . After squaring to account for self-attention, the training complexity scales as

$$\mathcal{C}_{\text{train}} = \mathcal{O}(n^4), \quad (6.15)$$

like VAR, but with a constant factor increase due to the doubled sequence length.

For generation, using sequential generation at scale s_k with sequence length $2n_k^2$ tokens, the self-attention operation over T time steps is

$$(2n_k^2)^2 T = 4n_k^4 T = 4a^{4(k-1)} T. \quad (6.16)$$

Summing over all K scales gives the total generation cost:

$$\mathcal{C}_{\text{gen}} = \sum_{k=1}^K 4a^{4(k-1)} T = 4T \sum_{k=1}^K a^{4(k-1)} = 4T \frac{a^{4K} - 1}{a^4 - 1}. \quad (6.17)$$

Substituting $a^K = a \cdot n$ yields

$$\mathcal{C}_{\text{gen}} = 4T \frac{a^4 n^4 - 1}{a^4 - 1} \sim \mathcal{O}(n^4 T), \quad (6.18)$$

showing that generation scales as $\mathcal{O}(n^4 T)$.

This indicates that the model, even with hierarchical scaling, does not depend on K , the number of scales, and has the same asymptotic complexity as standard diffusion models, with a reduced complexity in terms of sequence length compared to other autoregressive image generation models (excluding VAR).

Complexity Considerations: Our masked generation model differs from VAR in that teacher forcing requires concatenating the original sequence with the mask, effectively doubling the sequence length during training. Because attention scales quadratically, this leads to a theoretical $2^4 = 16$ -fold increase in operations relative to the base sequence. During inference, generation complexity similarly includes both the masked sequence and the number of iterative steps T , which further multiplies the computational cost.

Memory Considerations: Memory usage scales linearly with the number of tokens stored in attention, gradients, and optimiser states. For our masked model, storing both the original and masked sequences effectively doubles the required memory per batch. However, the impact extends beyond simple token storage. Attention matrices scale quadratically with sequence length, meaning that doubling the sequence length from m to $2m$ results in attention matrices that are $4\times$ larger.

For a sequence of length m , each token represented with b bits, and batch size B , the basic memory footprint is approximately

$$\text{Memory}_{\text{tokens}} \approx 2 \cdot B \cdot m \cdot b \text{ bits.}$$

The attention matrices contribute an additional quadratic term:

$$\text{Memory}_{\text{attention}} \approx B \cdot L \cdot H \cdot (2m)^2 \cdot p \text{ bits} = 4 \cdot B \cdot L \cdot H \cdot m^2 \cdot p \text{ bits,}$$

where L is the number of layers, H is the number of attention heads, and p is the precision in bits per attention weight. Similarly, key-value caches, gradient matrices, and intermediate activations all experience this quadratic scaling penalty.

For example, with a base sequence length of 680 tokens (typical for VAR), our approach requires storing attention matrices for 1360 tokens, increasing memory usage from $680^2 \approx 462\text{K}$ to $1360^2 \approx 1.85\text{M}$ attention weights per head per layer—a $4\times$ increase. This has practical implications for experimentation, as larger batch sizes or longer sequences can quickly exceed GPU memory, necessitating gradient checkpointing or reduced precision.

Chapter 7

The Repository

This chapter discusses the implementation of DiffusionVAR, detailing the architectural design, core model components, training infrastructure, and evaluation framework developed for this research.

7.1 System Architecture and Design

The DiffusionVAR repository implements a unified framework that extends the original VAR (Vector Autoregressive) architecture by incorporating diffusion processes for image generation. The system maintains backward compatibility with standard autoregressive generation whilst introducing diffusion-based training and inference methods through carefully designed modular components.

Main Changes From the VAR Repository The original VAR codebase was already well-structured and modular. One architectural change introduced, was the separation of the transformer backbone into a dedicated module (`transformer.py`), shared by both VAR and DiffusionVAR. This reduced redundancy and allowed the transformer to be used independently of model-specific logic. In the original VAR codebase, transformer functionality was embedded directly within each model definition, creating coupling that complicated extension. The refactored design extracts the transformer backbone into a reusable module, allowing both VAR and DiffusionVAR to share identical attention mechanisms, normalisation strategies, and positional encodings. This separation reduces redundancy, improves consistency across model variants, and simplifies future development. Model-specific functionality remains within dedicated modules: standard VAR logic in `models/dvar/var.py`, and diffusion-specific components in `models/dvar/dvar.py`. This organisation preserves the behaviour of the original VAR implementation while ensuring compatibility with the wider repository and aligning their implementation.

DiffusionVAR System Pipeline The model initialisation process employs a factory pattern implemented through the `build_vae_var` function in `models/__init__.py`, which serves as the primary entry point for model instantiation. This centralised approach enables dynamic model selection based on configuration parameters whilst maintaining consistent initialisation procedures across different model types. The initialisation sequence begins when `train.py` calls `build_vae_var` with the algorithm specification (`args.algo`). The factory function determines the appropriate model class through `get_model_class`, which performs conditional imports based on the algorithm parameter. For `algo='diffusion-var'`, the system imports `DiffusionVAR` from `models/dvar/dvar.py`; for `algo='var'`, it imports the standard VAR class from `models/dvar/var.py`.

Both model classes inherit from a common transformer backbone, ensuring architectural consistency whilst enabling algorithm-specific customisations. The backbone initialisation includes parameter-specific weight initialisation strategies, such as adaptive layer normalisation (AdaLN) configuration, attention normalisation settings, and head initialisation parameters. The transformer is passed a `zero_init` argument such that AdaLN layers to be initialised with zeros for DiffusionVAR, a strategy commonly employed in DiT models, but not used in standard VAR. This shared initialisation process eliminates code duplication whilst maintaining the flexibility to accommodate different model requirements. The VQVAE component initialisation follows a parallel process, loading pre-trained checkpoints. The system validates checkpoint compatibility and applies appropriate device placement.

The `utils/` directory provides essential infrastructure. This includes mixed precision optimisation (`amp_sc.py`), configuration management (`arg_util.py`), data loading (`data.py`), distributed sampling (`data_sampler.py`), image quality metrics (`image_metrics.py`), learning rate scheduling (`lr_control.py`), logging and checkpointing (`misc.py`), and experiment tracking (`wandb_setup.py`). The `visualisations/` directory contains tools for analysing model behaviour, including generation analysis, masking patterns, schedule visualisation, and VQVAE tokenisation analysis. Configuration files live in `config/`, with separate YAML files defining hyperparameters for VAR and DiffusionVAR models.

Architectural Philosophy The system architecture is designed around modularity, enforcing a clear separation of concerns across tokenisation, generation, and training. This allows components to be developed and tested independently while maintaining consistency through well-defined interfaces. Flexibility is achieved through a unified interface that supports multiple generation algorithms, diffusion scheduling and configurable files.

Software Engineering Principles The implementation follows established software engineering principles to maximise maintainability and extensibility. The DRY (Don't Repeat Yourself) principle is enforced through the shared transformer backbone and unified initialisation procedures, removing redundancy across model variants whilst ensuring consistency. Consistent interfaces across model types allow seamless switching between algorithms without altering training or evaluation pipelines.

Each directory serves a specific purpose with clear boundaries between components, allowing developers to work on individual modules independently while maintaining system coherence. Consistent naming conventions with shape indications and code separation make the system easier to navigate and understand. Algorithm-specific details are abstracted, with specialised features accessible on demand. The codebase adopts US spelling conventions to remain consistent with the wider Python and PyTorch framework.

7.2 Training and Model Checkpointing

The modular training framework, implemented in `trainer.py`, adopts a unified design that supports both the original VAR and the proposed DiffusionVAR architectures within a single codebase. Retaining the VAR implementation provides a consistent experimental baseline, enabling direct comparison against diffusion-augmented variants under identical training protocols. The `Trainer` class sets the algorithm through the `algo` parameter, allowing easy switching between training modes without code duplication. The trainer architecture retains support for progressive training via careful state management of the `prog_si` (progressive stage index) and `prog_wp` (progressive warmup) parameters. Although this functionality was not actively experimented with, it was preserved. Progressive

training allows models to gradually increase in complexity, focusing on lower scales first then incorporating later ones later.

Checkpointing and state management in the training framework enable resumption of experiments from the most recent valid state, which is essential for long-running or resource-constrained runs. The system maintains multiple checkpoints: the latest checkpoint, the best-performing checkpoint, determined by the validation loss, and the penultimate checkpoint to safeguard against potential corruption during saving, changing the renaming the last saved checkpoint before saving the next. Automatic resumption functionality detects existing checkpoints in the output directory and continues training from the most recent available state, ensuring reliability on shared computing resources subject to interruptions.

Mixed-precision training uses the `AmpOptimizer` class to reduce memory usage and speed up computation by combining 16-bit and 32-bit floating-point operations. The system handles the complexity of switching between precisions automatically. During training, the loss is adjusted for gradient accumulation to ensure proper averaging. When using mixed precision, the loss is scaled up before backpropagation to prevent gradients from becoming too small and disappearing in 16-bit arithmetic. The optimiser applies gradient clipping to keep training stable by controlling how large the gradients can become. After computing gradients, the system rescales them back to their original size, applies the optimiser update. Finally, gradients are cleared from memory to prepare for the next training step.

The `train.py` script is structured around three core functions: `build_everything`, `train_one_ep`, and `main_training`, each responsible for a distinct stage in the training pipeline. `build_everything` handles initialisation and setup. It loads configuration parameters, constructs data loaders, builds the VQ-VAE and VAR or DiffusionVAR models, and initialises the optimiser and trainer. Models are wrapped in distributed data parallelism (DDP) when running in a multi-GPU environment, or a neutral wrapper is used otherwise. This stage also supports PyTorch’s model compilation to reduced overhead or autotuning. `train_one_ep` iterates over each batch in the dataset, performing forward and backward passes and updating model parameters. During training, metrics are periodically logged to TensorBoard and Weights & Biases (wandb) to monitor progress. `main_training` orchestrates the overall training process, managing epoch level control and statistics. It initialises logging, iteratively calls `train_one_ep` for each epoch, and records performance metrics to TensorBoard and wandb.

7.3 Data Pipeline

The data pipeline, implemented in `data.py`, is designed for efficient handling of large-scale image datasets, supporting TAR archives. TAR (Tape Archive) files store many images in a single archive, reducing filesystem overhead and improving I/O performance during training. Images are standardised to a target resolution, optionally augmented with flips and crops, and normalised to $[-1, 1]$. Sampling is handled via `data_sampler.py`. `EvalDistributedSampler` deterministically partitions validation data across distributed workers for reproducible evaluation, while `DistInfiniteBatchSampler` extends this to training, maintaining consistent global batch sizes, optional repeated augmentation, and last-batch filling. Both `InfiniteBatchSampler` and `DistInfiniteBatchSampler` implement reproducible shuffling and batch composition. Shuffling is seeded with the current epoch and a user-specified global seed, ensuring each worker sees a consistent sequence of indices across runs.

7.4 Configuration Management

The configuration management system was redesigned to support multiple experimental setups while preserving modularity and reproducibility. Moving from hardcoded argument parsing, with variation only through command line, to a flexible YAML-based system represents a key improvement in workflow efficiency and maintainability.

Initial Argument Parsing Limitations The original configuration system, implemented via the Tap library in `arg_util.py`, used class-based definitions with all hyperparameters hardcoded as attributes, allowing changes only by command line. While sufficient for single experiments, it proved inconvenient when running multiple experiments, tracking configurations or testing different configurations simultaneously. Comparative studies required duplicating argument classes in separate files, introducing maintenance overhead, violating the DRY (Don't Repeat Yourself) principle and where bug fixes or feature additions required synchronisation across multiple nearly identical files. This also complicated version control, as meaningful configuration differences were hidden within broader code changes, making it harder to track hyperparameter settings.

YAML-Based Configuration Architecture The redesigned `Args` class extends Tap functionality with a `from_yaml` method, allowing configuration instantiation directly from structured YAML files. This format supports hierarchical organisation of complex settings, such as diffusion parameters and model architectures, improving readability and enabling type checking and validation. The structured format enables comprehensive validation and type checking whilst maintaining readability. Users can specify a configuration file via the `-config` command-line argument, with command-line overrides taking precedence. This combination preserves the immediacy of interactive experimentation while maintaining systematic reproducibility. Researchers can maintain a library of baseline configurations and modify specific parameters as needed without duplicating files. Error handling ensures graceful degradation when configuration files are missing or malformed, providing informative messages and sensible defaults. The system remains compatible with the original Tap-based interface while offering enhanced flexibility, robustness, and maintainability for large-scale experimentation.

7.5 Distributed Training

Distributed training is implemented using PyTorch's `DistributedDataParallel` (DDP) to enable efficient multi-GPU training across single- and multi-node setups. The `dist.py` module abstracts process group initialisation, device management, and synchronisation primitives. Each GPU processes a distinct batch subset while gradients are synchronised via all-reduce operations, ensuring consistent model parameters across workers.

7.6 Experimental Monitoring and Logging

TensorBoard Integration TensorBoard logging, implemented through the `TensorboardLogger` class in `misc.py`, providing real-time monitoring of training metrics and model behaviour. The system logs training and validation losses, accuracy metrics, learning rate schedules, and gradient statistics with configurable frequency. Distributed training scenarios require coordination to ensure consistent logging from the master process whilst avoiding redundant log entries from worker processes.

Weights & Biases Integration Weights & Biases (WandB) integration, implemented in `wandb_setup.py`, provides visual experiment tracking. The system includes error handling and graceful degradation, ensuring uninterrupted training even if WandB services are unavailable. Training losses, validation results, and hyperparameters are logged to enable systematic cross-experiment comparison. Step numbering and virtual epochs maintain consistent temporal references across experiments with different batch sizes or accumulation strategies. I introduced scale-specific logging, recording loss and accuracy metrics at each scale independently, for analysis of how model performance evolves for the scales. Image logging and metric computation (FID, LPIPS, Inception Score) provide visual and quantitative feedback on generation quality. Comprehensive configuration tracking captures all training settings and model parameters. The system supports experiment resumption through run ID management, enabling continued logging after training interruptions whilst maintaining experimental continuity.

Chapter 8

Conclusion

8.1 Conclusion and Reflections

In this project, I developed a transformer-based model that combines elements of autoregressive (VAR) and masked diffusion architectures. Despite extensive experimentation and testing, the model did not surpass the original VAR architecture in terms of generative fidelity or computational efficiency. Nevertheless, the project demonstrates the feasibility of adapting autoregressive transformers for diffusion-based generation, and the results provide insights into how hierarchical token sequences can be conditioned on timesteps and class embeddings through zero-initialised AdaLN layers.

Had there been more time, I would have trained the model for longer to potentially improve convergence and generative quality. Additionally, a more thorough exploration of hyperparameter configurations—such as noise schedules, sampling temperatures. Evaluating image generation metrics on a per-scale basis would also be valuable, as different token maps carry varying amounts of information, which may require distinct modulation strategies across scales.

One direction for future work would be the exploration of scale-specific masking or conditioning schedules. In hierarchical token sequences, different scales contain varying numbers of tokens and convey different amounts of information. Applying a uniform conditioning or noise schedule across all scales may be suboptimal, as some scales may not benefit as much from global context or standard diffusion noise rates. By exploring schedules per scale or designing adaptive schedules that vary according to the scale, the model could potentially learn more effectively. Such scale-aware conditioning could improve both training stability and generative fidelity, as the model would more appropriately allocate capacity and attention to the information present at each hierarchical level. Specifically, coarse-scale token maps represent global structure and semantic content, suggesting they would benefit from early denoising steps with lower noise levels to preserve structural coherence. Conversely, fine-scale maps encode detailed textures and local features that could tolerate higher noise levels initially, allowing for more exploratory sampling before converging to refined details. This scale-dependent approach could be implemented through an adaptive noise schedule where the variance schedule σ_t varies per scale based on information density.

Another avenue for architectural refinement involves reconsidering the learned mask embedding strategy currently employed for representing masked tokens. The current approach utilises a single learnable embedding vector that replaces all masked tokens during the forward pass, which may inadvertently introduce unwanted inductive biases or limit the model’s representational flexibility. One potential improvement would be to eliminate this learned mask embedding entirely and instead employ positional or contextual encodings that preserve spatial relationships without introducing a fixed representational

bottleneck. An alternative approach could involve replacing the static mask embedding with dynamic, context-dependent representations generated through stop-gradient mechanisms. In this framework, the masked token representations could be derived from the surrounding unmasked context through a lightweight prediction network, with gradients stopped to prevent the model from trivially learning to reconstruct rather than generate. This stop-gradient approach would ensure that the context-derived representations serve purely as positional anchors without allowing the model to exploit reconstruction shortcuts that could undermine the generative objective. The implementation would involve computing contextual embeddings from neighbouring unmasked tokens, applying a stop-gradient operation to prevent backpropagation through these computed representations, and using these frozen context vectors as more informative mask tokens. However, the applicability of stop-gradient techniques in this context requires careful consideration of the training dynamics. Unlike contrastive learning scenarios where stop-gradients prevent representation collapse, the masked autoregressive setting may not benefit equally from such constraints. The effectiveness would largely depend on whether the contextual prediction network can generate sufficiently informative representations without access to the target token information, and whether preventing gradient flow through these representations improves rather than hinders the overall learning objective. A more conservative approach might involve learnable mask embeddings that are periodically reset or regularised to prevent over-fitting to specific token patterns, maintaining the benefits of learnable representations while avoiding potential representational stagnation.

Legal, Social, Ethical, and Professional Considerations. This project involved developing a novel model architecture and experimenting with different transformer and diffusion components. No direct legal, social, or ethical issues arose from the work itself. The project did, however, require substantial computational resources, with training taking approximately 7.5 hours per model on two GPUs, and numerous configurations tested to assess stability and performance. Professional responsibility was maintained throughout by documenting experiments carefully, validating results, and acknowledging prior work appropriately.

AI Declaration. AI tools were utilised in a limited and responsible manner throughout this project. Specifically, AI was employed to help identify relevant literature, clarify technical explanations, assist with debugging, and aid in formatting LaTeX sections for clarity, including generating properly formatted citations. Additionally, AI was used as a reference to better understand complex codebases and existing implementations, providing guidance on structure and functionality without directly altering the code. The core development, experimental design, coding, and analysis were performed independently. AI support was strictly advisory and did not contribute directly to the generation of model architectures, experimental results, or primary research insights.

Bibliography

- [1] Marro S, Evangelista D, Huang XA, La Malfa E, Lombardi M, Wooldridge M. Language Models Are Implicitly Continuous. arXiv preprint arXiv:250403933. 2025 apr.
- [2] van den Oord A, Kalchbrenner N, Kavukcuoglu K. Conditional image generation with PixelCNN decoders. In: Advances in neural information processing systems; 2016. p. 4790-8.
- [3] van den Oord A, Vinyals O, Kavukcuoglu K. Neural Discrete Representation Learning. arXiv preprint arXiv:171100937. 2017.
- [4] Brown TB, Mann B, Ryder N, Subbiah M, et al. Language models are few-shot learners. Advances in neural information processing systems. 2020;33:1877-901.
- [5] Touvron H, Lavril T, Izacard G, et al. LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:230213971. 2023.
- [6] Razavi A, van den Oord A, Vinyals O. Generating diverse high-fidelity images with VQ-VAE-2. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 32; 2019. p. 14837-47.
- [7] Esser P, Rombach R, Ommer B. Taming Transformers for High-Resolution Image Synthesis. CVPR. 2021.
- [8] Devlin J, Chang MW, Lee K, Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:181004805. 2019.
- [9] Chang H, Zhang H, Jiang L, Liu C, Freeman WT. MaskGIT: Masked generative image transformer. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2022. p. 11315-25.
- [10] Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems. 2020;33:6840-51.
- [11] Rombach R, Blattmann A, Lorenz D, Esser P, Ommer B. High-resolution image synthesis with latent diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2022. p. 10684-95.
- [12] Betker E, OpenAI. DALL·E 3; 2023.
- [13] Tian K, Jiang Y, Yuan Z, Peng B, Wang L. Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction. arXiv preprint arXiv:240402905. 2024.
- [14] Parmar N, Vaswani A, Uszkoreit J, et al. Image Transformer. In: International Conference on Machine Learning; 2018. p. 4055-64.
- [15] Dhariwal P, Nichol A. Diffusion models beat GANs on image synthesis. In: Advances in Neural Information Processing Systems. vol. 34; 2021. p. 8780-94.

- [16] Esser P, Rombach R, Ommer B. Taming transformers for high-resolution image synthesis. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2021. p. 12873-83.
- [17] Yu J, Li X, Koh JY, Zhang H, Pang R, Qin J, et al. Vector-quantized image modeling with improved VQGAN. arXiv preprint arXiv:211004627. 2021.
- [18] Song J, Meng C, Ermon S. Denoising Diffusion Implicit Models. In: ICLR; 2021. .
- [19] Song Y, Sohl-Dickstein J, Kingma DP, Kumar A, Ermon S, Poole B. Score-based generative modeling through stochastic differential equations. In: International Conference on Learning Representations; 2021. .
- [20] Lu C, Luo Y, Kong Z, Telyakovskiy T, Chen Q, Wu YN. DPM-Solver++: Fast solver for diffusion probabilistic models. arXiv preprint arXiv:221101095. 2022.
- [21] Sun H, Yu L, Dai B, Schuurmans D, Dai H. Score-based Continuous-time Discrete Diffusion Models. arXiv preprint arXiv:221116750. 2022.
- [22] Sahoo SS, Arriola M, Schiff Y, Gokaslan A, Marroquin E, Chiu JT, et al. Simple and Effective Masked Diffusion Language Models. arXiv preprint arXiv:240607524. 2024.
- [23] Shi J, Han K, Wang Z, Doucet A, Titsias MK. Simplified and Generalized Masked Diffusion for Discrete Data. arXiv preprint arXiv:240604329. 2024.
- [24] Li T, Tian Y, Li H, Deng M, He K. Autoregressive Image Generation without Vector Quantization. arXiv preprint arXiv:240611838. 2024.
- [25] Sohl-Dickstein J, Weiss EA, Maheswaranathan N, Ganguli S. Deep unsupervised learning using nonequilibrium thermodynamics. arXiv preprint arXiv:150303585. 2015.
- [26] Hur J, Lee DJ, Han G, et al. Unlocking the capabilities of masked generative models for image synthesis via self-guidance. arXiv. 2024.
- [27] You Z, Ou J, Zhang X, et al. Effective and efficient masked image generation models. arXiv. 2025.
- [28] Liu X, Zhang X, Zhou J, et al. Plug-and-play context feature reuse for efficient masked generation. arXiv. 2025.
- [29] Austin J, Johnson DD, Ho J, Tarlow D, Van Den Berg R. Structured denoising diffusion models in discrete state-spaces. In: Advances in Neural Information Processing Systems; 2021. .
- [30] Zhao L, Ding X, Yu L, Akoglu L. Improving and unifying discrete and continuous-time discrete denoising diffusion. arXiv preprint arXiv:240203701. 2024.
- [31] Ou J, Nie S, Xue K, Zhu F, Sun J, Li Z, et al. Your Absorbing Discrete Diffusion Secretly Models the Conditional Distributions of Clean Data. arXiv preprint arXiv:240603736. 2024.
- [32] Peebles W, Xie JY. Scalable diffusion models with transformers. In: ICCV; 2023. .
- [33] Nichol A, Dhariwal P, Ramesh A, Shyam P, Mishkin P, McGrew B, et al. GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models. arXiv preprint arXiv:211210741. 2021.

- [34] Kong Z, Ping W, Huang J, Zhao K, Catanzaro B. DiffWave: A Versatile Diffusion Model for Audio Synthesis. arXiv preprint arXiv:200909761. 2020.
- [35] Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I. Language models are unsupervised multitask learners. OpenAI blog. 2019;1(8).
- [36] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention Is All You Need. In: Advances in Neural Information Processing Systems; 2017. p. 5998-6008.
- [37] Ho J, Salimans T. Classifier-Guided Diffusion Sampling. In: Advances in Neural Information Processing Systems; 2021. .
- [38] Gugger S, Howard J. Hands-On Generative AI with Diffusion Models. O'Reilly Media; 2024.
- [39] Nichol AQ, Dhariwal P, Ramesh A, Shyam P, Mishkin P, McGrew B, et al. GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models. In: Proceedings of the 39th International Conference on Machine Learning (ICML). PMLR; 2022. p. 16784-804.
- [40] Guo Z, Lang J, Huang S, Gao Y, Ding X. A Comprehensive Review on Noise Control of Diffusion Model. arXiv preprint arXiv:250204669. 2025.
- [41] Nichol A, Dhariwal P. Improved Denoising Diffusion Probabilistic Models. arXiv preprint arXiv:210209672. 2021.
- [42] Jabri A, Fleet D, Chen T. Scalable Adaptive Computation for Iterative Generation. arXiv preprint arXiv:221211972. 2022.
- [43] Hoogeboom E, Nielsen D, Jaini P, Forré P, Welling M. Argmax Flows and Multinomial Diffusion: Learning Categorical Distributions. In: International Conference on Machine Learning; 2021. .
- [44] Zhang L. The Cosine Schedule is Fisher-Rao-Optimal for Masked Discrete Diffusion Models. arXiv preprint arXiv:250804884. 2025.
- [45] Luxembourg O, Permuter H, Nachmani E. Plan for Speed: Dilated Scheduling for Masked Diffusion Language Models. arXiv preprint arXiv:250619037. 2025.
- [46] He H, Renz K, Cao Y, Geiger A. MDPO: Overcoming the Training-Inference Divide of Masked Diffusion Language Models. arXiv preprint arXiv:250813148. 2025.
- [47] Arriola M, Gokaslan A, Chiu JT, Yang Z, Qi Z, Han J, et al. Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models. In: Proceedings of the Thirteenth International Conference on Learning Representations (ICLR); 2025. .
- [48] Peebles W, Xie S. Scalable Diffusion Models with Transformers. arXiv preprint arXiv:221209748. 2022.
- [49] Dao T, Fu DY, Ermon S, Rudra A, Ré C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv preprint arXiv:220514135. 2022.
- [50] Guo Y, Wang C, Yu SX, McKenna F, Law KH. AdaLN: A Vision Transformer for Multidomain Learning and Predisaster Building Information Extraction from Images. Journal of Computing in Civil Engineering. 2022.

- [51] Ho J, Salimans T. Classifier-Free Diffusion Guidance. arXiv preprint arXiv:2207.12598. 2022.
- [52] Pope R, Douglas S, Chowdhery A, Devlin J, Bradbury J, Levskaya A, et al. Efficiently Scaling Transformer Inference. arXiv preprint arXiv:2211.05102. 2022.